Evolving Systems (2023) 14:319–341
https://doi.org/10.1007/s12530-022-09455-z

ORIGINAL PAPER

Evolving fuzzy neural classifier that integrates uncertainty from human-expert feedback

Paulo Vitor de Campos Souza[1] · **Edwin Lughofer**[1]

Received: 13 January 2021 / Accepted: 17 July 2022 / Published online: 15 August 2022
© The Author(s) 2022

Abstract
Evolving fuzzy neural networks are models capable of solving complex problems in a wide variety of contexts. In general, the quality of the data evaluated by a model has a direct impact on the quality of the results. Some procedures can generate uncertainty during data collection, which can be identified by experts to choose more suitable forms of model training. This paper proposes the integration of expert input on labeling uncertainty into evolving fuzzy neural classifiers (EFNC) in an approach called *EFNC-U*. Uncertainty is considered in class label input provided by experts, who may not be entirely confident in their labeling or who may have limited experience with the application scenario for which the data is processed. Further, we aimed to create highly interpretable fuzzy classification rules to gain a better understanding of the process and thus to enable the user to elicit new knowledge from the model. To prove our technique, we performed binary pattern classification tests within two application scenarios, cyber invasion and fraud detection in auctions. By explicitly considering class label uncertainty in the update process of the EFNC-U, improved accuracy trend lines were achieved compared to fully (and blindly) updating the classifiers with uncertain data. Integration of (simulated) labeling uncertainty smaller than 20% led to similar accuracy trends as using the original streams (unaffected by uncertainty). This demonstrates the robustness of our approach up to this uncertainty level. Finally, interpretable rules were elicited for a particular application (auction fraud identification) with reduced (and thus readable) antecedent lengths and with certainty values in the consequent class labels. Additionally, an average expected uncertainty of the rules were elicited based on the uncertainty levels in those samples which formed the corresponding rules.

Keywords Evolving fuzzy neural classifier · Class label uncertainty · User annotation feedback · Stream classification · Interpretability of fuzzy classification rules

1 Introduction

Evolving models are intelligent approaches capable of working with data online: They update their parameters dynamically and incrementally based on new incoming data, typically in form of a sequential data stream (Gama 2010), and can therefore tackle on-line learning problems (Kasabov 2007). All intelligent models have specific advantages and disadvantages. Combining two or more intelligent techniques allows leveraging their advantages to produce more accurate solutions to the problem at hand (de Campos Souza 2020).

Evolving fuzzy (neural) classifiers (EF(N)C) are able to extract granules in the form of rules that contain IF-THEN connections in their antecedent parts and embed linguistically readable terms in order to explain their consequents, which are represented by class labels. Thus, compared to other machine learning (ML) classifiers, they offer interpretability in the form of input feature interrelations for particular classes and may allow an advanced form of user interaction directly with the model. Pioneering work on how to establish and update such classifiers in an evolving context from streams was presented in Angelov et al. (2008), with several extensions proposed in Angelov and Zhou (2008), Lughofer and Buchtala (2013), Wang et al. (2013) or Angelov and Gu (2018) (the later for deep learning aspects);

✉ Paulo Vitor de Campos Souza
paulo.de_campos_souza@jku.at

Edwin Lughofer
edwin.lughofer@jku.at

[1] Department of Knowledge-Based Mathematical Systems, Johannes Kepler Universitat Linz, Science Park 2 (6th Floor), Altenbergerstrasse 69, 4040 Linz, Austria

see Škrjanc et al. (2019) for a comprehensive survey. Accuracy and interpretability of evolving fuzzy neural classifiers depend heavily on the quality of the data they are fed to perform pattern classification tasks. In general, data collection processes should meet standardized scientific criteria to avoid distortions in the results produced (Angelov and Zhou 2008). However, some devices, procedures, or processes for data collection may be subjected to external influences that can generate uncertainty in the data (Hariri et al. 2019). Models that work, for instance, with robot controllers (Se et al. 2002), face recognition (Xu et al. 2014) and the internet of things in medical contexts (Al-Turjman et al. 2019) can be affected by uncertainty in the data collection. Generally, experts can identify distortions and assign a degree of uncertainty to individual samples collected (Solomatine and Shrestha 2009).

Data uncertainty has been thoroughly addressed in machine learning models (Ghahramani 2015), and in particular in evolving approaches in Leite et al. (2012, 2013), Pratama et al. (2016), Subramanian et al. (2014). The approaches in Leite et al. (2012, 2013) consider uncertainty in the rules' consequents by providing an interval-based output value in addition to Takaki–Sugeno (TS)-like hyper-planes. In the approaches in Pratama et al. (2016) and Subramanian et al. (2014), a 'higher level' of uncertainty is addressed by type-2 fuzzy sets, which are able to represent the uncertainty in the definitions of the fuzzy sets themselves (and thus in the local granular parts these represent in the feature space). These approaches, however, address uncertainty in the form of measurement noise or natural variation of the (target) sample values. Furthermore, for the input feature values, uncertainty is addressed by evolving fuzzy (neural) classifiers in a straightforward way due to the automatic extraction of fuzzy sets (Hühn and Hüllermeier 2009). None of the evolving approaches published to date addresses uncertainty explicitly when provided by the user along the particular target (class) labels (e.g., as a real value in [0, 0.5], with 0 indicating no uncertainty in the class label, and 0.5 indicating full uncertainty)—they extract the parameters/structures in their models solely based on measurement variations and the noise content.

1.1 Our approach

The main objective of this work was to develop an evolving fuzzy neural classifier that incorporates user feedback in the form of label uncertainty of the samples fed to the model by considering explicitly this uncertainty when updating and evolving the classifier. We integrated uncertainty levels within the range [0, 0.5] (i) in the update of the data cloud structure by extending the classical (unsupervised) SODA (Self-Organized Direction Aware) approach (Gu and Angelov 2018) to create a supervised SODA with uncertainty integration (termed as *SSODA-U*), based on which the fuzzy neurons are constructed by using n-uninorms (rather than classical t-norms as commonly used in ENFS approaches (Škrjanc et al. 2019)), and (ii) in the update of the consequent parameters using recursive least squares (RLS), which we extended to create an uncertainty-based RLS approach.

The first layer of the model features a fuzzification method capable of working with online samples and of updating the parameters of data clouds (i.e., their centers and standard deviation) which are formed based on the (local) data density. We also propose a supervised evaluation of the clouds' identification with the model's expected output in order to (a) find the best cloud to which a new sample should be assigned or (b) alternatively evolve a new cloud. Furthermore, class label uncertainty levels are used when updating the clouds and in the clouds' evolution process. In the second layer of the model, the concept of n-uninorms allows the aggregation of all neurons in the first layer to build advanced IF-THEN fuzzy rules, not just allowing AND-connections between the antecedent parts (and features contained therein), but also OR-connections, which may greatly increase the flexibility in the interpretation of the classification rules. The rules also help to define the weights that connect the first two layers of the model (fuzzy inference system) with the neural aggregation network responsible for the model's defuzzification process and, consequently, the model's final outputs. These weights are learned incrementally by a modified RLS approach which integrates the uncertainty levels in the update.

The main highlights of our new approach are:

- Construction of a model that is capable of working online and updates its model hyper-parameters based on user/expert uncertainty information.
- Identification of the relevance of user input to the essence of the data and, consequently, the construction of a fuzzification process which is consistent with the data's real behavior.
- Usage of an approach based on data density to build neurons from classification clusters, which is extended for considering label uncertainty in the cloud update and forming process.
- Extraction of advanced, interpretable rules that allow AND- and OR-connections with uncertainty information in the consequent labels and in the whole rule, depending on the uncertainty levels in the data used for updating the rules (clouds).
- An approach for decreasing the length of the rules (thus increasing their readability) by down-weighting input features which are less relevant to explain the stream classification problem.

The paper is composed as follows. Section 2 describes several state-of-the-art approaches for handling data uncertainty and user input, and how they work effectively in machine learning models. Section 3 presents the main theoretical concepts that underlie our model's key components. Further, it describes the evolving fuzzy neural network architecture, the fuzzy neuron, and all aspects related to the incremental evolving training procedures of our evolving neuro-fuzzy classifier, and how uncertainty in user feedback is used for the construction of the model architecture and for model training. In Sect. 4, the results are compared to those of related SoA approaches, and the interpretation of the rules obtained is demonstrated. The results showed that considering class label uncertainty explicitly in the updating process of the EFNC could improve the accuracy trend lines, compared to fully (and blindly) updating the classifiers with uncertain labels in the data. Furthermore, we found that the integration of (simulated) label uncertainty below 20% still led to similar accuracy trends as using the original streams (unaffected by uncertainty). This demonstrates the robustness of our approach up to this uncertainty level. Finally, Sect. 5 concludes the paper.

2 Literature review

2.1 Data uncertainty

There are some external and internal uncertainties in the data collection process. These procedures can be influenced by human inference, computational procedures, or equipment used for data acquisition (Chatfield 1995) (e.g., due to measurement noise).

The process of measuring physical phenomena is a transfer of information from a source system to an operator via a measurement system. The interaction between a source system and a measurement system causes modification to both, which is coded in a vector for transferring the desired information. Consider, for example, the problem of measuring the temperature of a small body in a water reservoir by means of an encapsulated sensor element or a liquid inside glass. Even if the measurement system does not significantly change the source system, it must reach equilibrium with it, and the measurement may not have taken into account all relevant information phenomena. As a result, the sensor may indicate a reading that does not capture the desired phenomenon (Dong et al. 2009).

The physical quantity obtained in an experimental procedure is always an approximation of its real value. The theory of errors aims to determine the best possible value for the quantity and how much it can differ from the correct value. The best possible value is also called the best estimate or the experimental value of the measurand. Uncertainty can then be defined as an indication of how much the best value may differ from the actual value. In other words, uncertainty is an estimate of the error, that is, the error's value if it could be measured (Chatfield 1995).

It is often the case in experiments that uncertainty is greater than the instrument's error limit. This happens, for example, when the variable to be measured exhibits an intrinsically varying behavior. To determine the uncertainty of an experimental data set, some approaches can be used. Uncertainty and least squares regression have been addressed by the authors in Hodges and Moore (1972). The authors in Bi et al. (2005) modeled uncertainty for support vector models in classification problems. The work in Huijbregts et al. (2001) dealt with aspects of uncertainty in life cycle inventory problems, and the authors in Kirk and Stumpf (2009) assessed the impact of data uncertainty on Gaussian process regression models. To define the parameters of a problem, fuzzy neural networks use numerical data involving data collection. Since fuzzification processes transform data into fuzzy sets, a high degree of uncertainty in this data impacts the construction of the model architecture. In this scenario, it is essential to consider uncertainty in the construction of the model parameters and the fuzzy neurons (Schnute 1987). In fuzzy systems, fuzzy neural networks and fuzzy classifiers, this is typically achieved in a straightforward way Hühn and Hüllermeier (2009) during the formation, extraction or definition of the rule antecedent parts, which contain fuzzy sets (Zadeh 1965); these have a particular support for a part of the feature space (typically represented in the form of a linguistic term (Gacto et al. 2011)), and thus the wider the support becomes, the more uncertain the information contained in data set (Siler and Buckley 2005). Handling of uncertainty in the class labels has been loosely addressed in batch construction of fuzzy systems (see D'angelo et al. (2015) and Casalino et al. (2019)), and has, to the best of our knowledge, remained unaddressed in evolving fuzzy systems/classifier approaches.

2.2 Clustering of data streams with uncertainty

Clustering plays an important role in our approach, as it provides the granulation of the input feature space into several clusters (each cluster (in the form of a cloud) is associated with a neuron, see subsequent section, and constructed using n-uninorms). The procedures applied use several ways of defining how the data will be grouped (clustered). In most approaches, the distance between samples is one of the most relevant criteria for defining the proximity to a cluster center (Zhang and Pal 2000; Pedrycz and Izakian 2014). Construction of fuzzy neural network architectures already uses fuzzy c-means Lemos et al. (2010), axiomatic fuzzy sets (Duan et al. 2018), approaches based on data density (Guimarães et al. 2020; Souza et al. 2020; Zhou et al. 2020; de Campos

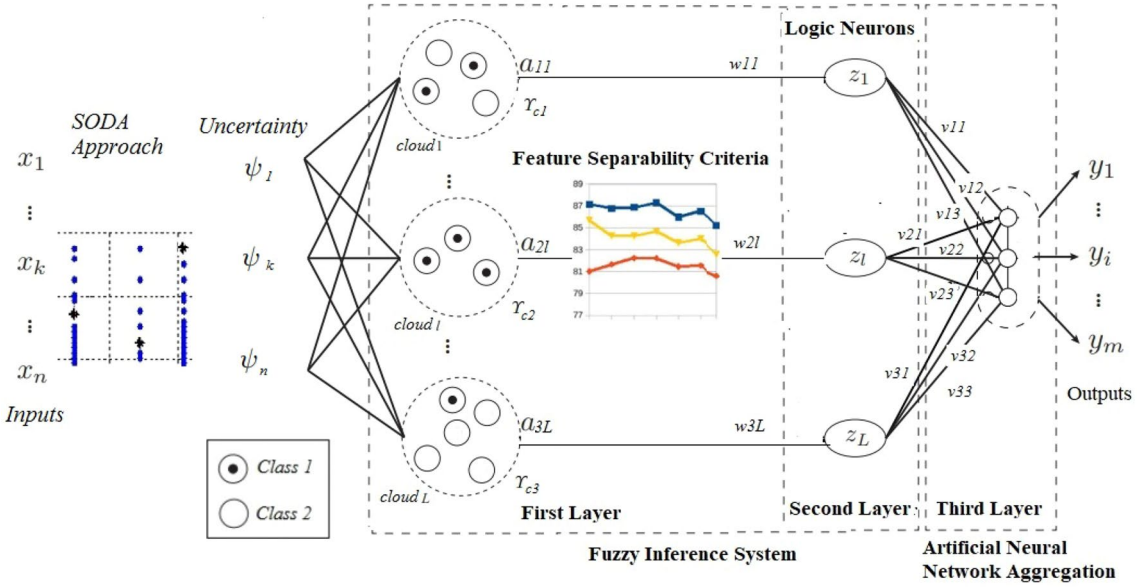


Fig. 1 EFNC-U architecture

Souza et al. 2021a, b, c; de Campos Souza and Lughofer 2022b), polynomial nodes Zhang et al. (2020), Bayesian approaches (Souza et al. 2019b, 2021), and equally spaced membership functions (Souza 2018; Souza et al. 2019a, 2020).

In recent years, factors such as data uncertainty have been incorporated into the criteria for defining or updating data clusters. In this context, the factors that involve the uncertainty of samples can define the best allocation of a new sample within a cluster (Hamidzadeh and Ghadamyari 2020). The authors in (Hamidzadeh and Ghadamyari 2020) addressed several clustering techniques that consider data uncertainty, highlighting their techniques and roles and possible applications. Approaches such as CluStream (Aggarwal et al. 2003), DenStream (Cao et al. 2006), and D-Stream (Chen and Tu 2007) stand out, which have online and offline phases to work with stream data and to build clusters incrementally. Therefore, the incorporation of data uncertainty into the fuzzification process has been consistently addressed in the recent years (Hamidzadeh and Ghadamyari 2020). However, here we address uncertainty that is based on user annotation expertise or explicit user input and which is not calculated from the data—as handled by all the approaches mentioned above. Data uncertainty has recently been addressed by approaches described in Ge et al. (2020). However, unlike in our work, data uncertainties are evaluated based on random values in a time-delay model and not on user-related uncertainty during the collection or the evaluation of new information.

3 An evolving fuzzy neural classifier architecture that considers uncertainty: structure and training approach

Our approach helps in particular the model's architecture to define its logical structure (Kasabov 2001). The evolving model described in this paper supports most of the structure defined in de Campos Souza et al. (2019), mainly in the model's architecture and the construction of Gaussian neurons. However, adjustments were made in the first layer (fuzzification) and in the model's training process to include uncertainty levels in class labels and to achieve a more efficient update in the case of higher uncertainty. Unlike in our previous model, the first layer's weights are not defined in an aleatory way. A process that increases interpretability of the fuzzy rules is incorporated to define the Gaussian weights based on the feature importance levels for the classification problem at hand. Another significant difference from the approach in de Campos Souza et al. (2019) is that no pruning criteria are applied to the fuzzy neurons in the second layer, but a control approach over the clusters formed by the fuzzification process, preventing irrelevant clouds from contributing to the construction of the Gaussian neurons.

The model proposed in this paper has three layers in its architecture, where the first and the second are fuzzy inference based layers, and the third is a neural aggregation network layer responsible for defuzzification. Figure 1 illustrates the general feedforward topology of the evolving fuzzy neural classifier with uncertainty integration (EFNC-U) used in this paper. It uses the SSODA-U technique (as

an extension of the classical SODA) to create neurons in the first layer for constructing the fuzzy neurons in the second layer. The first two layers form a fuzzy inference system which can create fuzzy rules that extract knowledge from the problem being analyzed.

3.1 First layer

Essentially, the fuzzification process defines the first layer's L neurons. The data clustering approach defines the centers (c) and sigmas ($\boldsymbol{\sigma}$) of the clusters that are the basis for forming the Gaussian neurons in the first layer. For each input feature x_j, L fuzzy sets are represented by $A_i^j, j = 1, \ldots, L$, whose membership functions are the activation functions of the corresponding neurons. Therefore, the final results of the fuzzification layer are the membership degrees associated with the input values:

$$a_{ij} = \mu(A_i^j) \, for \, i = 1, \ldots, \, N \, and \, j = 1, \ldots, L \tag{1}$$

where N is the number of inputs, and L represents the number of fuzzy sets (neurons) for each input (extracted from each cloud in each dimension). Therefore, in the case of Gaussian neurons formed in the first layer, the membership degrees can be expressed by:

$$a_{ij}(x_i, c_{ij}, \sigma_{ij}) = e^{-\frac{1}{2}\left(\frac{x_i - c_{ij}}{\sigma_{ij}}\right)^2}, \, for \, i = 1 \ldots \, N, \, j = 1 \, \ldots \, L \tag{2}$$

where c_{ij} and σ_{ij} are the center and width of the jth Gaussian function for input variable i, respectively.

After constructing the Gaussian neurons, a weight vector w_{ij} (for $i = 1 \ldots N$ and $j = 1 \ldots L$)) is assigned to each of the L neurons in the first layer. They are usually defined at random in the range [0, 1]. However, we use an approach that extracts the weights based on the relationship between the input features and the classes in the stream classification problem. In particular, the weights are calculated to reflect how strong the discriminatory power of each feature among all classes is, based on a specified separability criterion. The feature weight proposal is based on a robust expansion of Fisher's separability criterion (Dy and Brodley 2000), which was originally used in Lughofer (2011). Our approach uses the criterion in (3) for estimating the significance of each feature to discriminate between the classes (Dy and Brodley 2004):

$$J = trace(S_w^{-1} S_b) \tag{3}$$

where *trace* defines the sum of the diagonal elements of the matrices S_w and S_b. S_b is the dispersion matrix between the classes, which measures the classes' dispersion averages to the total average of the elements, and S_w indicates the within-scatter matrix, which measures the samples' dispersion in their class averages (Lughofer 2011). The class separability is directly related to the value of J: higher values of J indicate a better separability between the classes with the used feature set in S_w^{-1} and S_b.

The approach presented in this paper uses each feature once, calculates S_w^{-1} and S_b, and finally determines (3) for each feature $\rightarrow J_1, \ldots, J_N$, with N features in the data set. In this technique, the higher the value of J_i, the more important i is for discriminating among the classes. Thus, feature weights are then assigned by:

$$\mathbf{w_j} = \frac{J_j}{\max_{i=1,\ldots,N} J_i} \tag{4}$$

yielding always a weight 1 for the most important feature and a value in [0, 1] in relation to it for the other features. This approach defines the weights of the neuron in the first layer such that they correlate with the dimensions of the problem analyzed and down-weight less important features in the neuron construction. The incremental adaptation of S_w^{-1} and S_b with new incoming samples (which may change the feature importance for the classification problem), which induces an update of J_i for each feature i by calculating (3), is described in detail in Lughofer (2011).

This type of procedure allows the neurons of the evolving fuzzy neural network to synchronize with the new data block's relevance to the dimensions, making the learning process smooth in the fuzzy classifiers, as the evolution of the weights changes smoothly and continuously over time. This approach can be seen as a technique for selecting the essential features, since dimensions irrelevant to the problem will have values close to zero, which leads to dimensionality reduction in a soft manner (Lughofer 2011); soft because features with low weights close to 0 will have a marginal impact (and not no impact at all when they become exactly 0, as is the case in classical hard dimension reduction approaches) and can therefore be ignored when showing the final rules to the user (also see the results section for a concrete example). Thus, the rule length can be reduced and transparency improved.

Defining the weights of Gaussian neurons based on the problem's features provides significant rules, since the impacts of rule antecedents will correlate with the resources they represent and will be directly linked to their ability to identify the problem's classes. Hence, antecedents with an impact close to zero can be seen as irrelevant, which in turn creates more compact rules and thus improves their readability. Due to the continuous update of these weights, variations in the problem's dimensions can be identified well; this means that features may become dynamically and autonomously reactivated due to new information in the data. This guarantees a smooth soft dimension reduction that does not require re-training phases with newly selected feature sets

(as it would be the case when performing updates for hard feature selection/ranking approaches).

3.1.1 Supervised self-organizing direction-aware data partitioning with class label uncertainty—SSODA-U

The approach used in this paper for the first-layer fuzzification process builds on the self-organizing direction-aware data partitioning approach and extends it with the integration of data uncertainty (SODA-U). Thereby, the original (unsupervised) technique (SODA) proposed in Gu and Angelov (2018) was adapted and extended to create a supervised approach and to integrate class label uncertainty adequately. More specifically, our fuzzification algorithm is based on data density in order to integrate the samples' class label uncertainty into the update of the clouds.

SODA is a data partitioning algorithm that builds on empirical data analysis (EDA) (Angelov et al. 2017b), which is commonly used to extract data clouds (clusters of arbitrary shape, which do not require any density function to be defined in advance) (Angelov and Yager 2012). These can then be used as a backbone for constructing fuzzy rules/neurons to achieve a suitable granulation of the input data space. SODA recognizes peaks/modes of the input frequency and applies them as focal points to join another feature to data clouds (similar to a Voronoi tessellation (Watson 1981)). The procedure for extracting data clouds is somewhat similar to clustering algorithms, with the main difference that they are non-parametric as they do not follow a pre-defined data distribution (Gu and Angelov 2018).

The method works with a magnitude component based on a universal distance metric and a directional/angular element based on the cosine similarity. The principal EDA operators were defined by the authors in Angelov et al. (2017b), suitable for processing data streams. We consider:

- R^m = real data space,
- $(x_1, x_2, \ldots, x_m)$ = data stream sample,
- $x_i = [x_{i1}, x_{i2}, \ldots, x_{im}]^T \in R^m$ = m-dimensional vector where $m = 1, 2, 3 \ldots$ is the dimensionality of the problem and i represents the time instance at which the ith data sample is processed.

Consider the data set analyzed in the nth instance of time denoted by $x_1, x_2, \ldots, x_n$, then a sorted group of distinct variables from $\mathbf{x}$ is given by $[\epsilon_{i1}, \epsilon_{i2}, \ldots, \epsilon_{im}]^T \in R^m$. Still, on the perception of consciousness associated to the clustering proposed by Gu and Angelov (2018), consider as normalized numbers of the replications $f_1, f_2, \ldots, f_{n_\epsilon}$ where n_ϵ (greater than one and smaller than n) is the number of unique data samples.

The SODA approach uses the Euclidean distance metric to define the magnitude component (M), which can be represented for two samples x_i and x_j by Gu and Angelov (2018):

$$d_M(x_i, x_j) = \left\| x_i - x_j \right\| = \sqrt{\sum_{k=1}^{m} (x_{ik} - x_{jk})^2} i, j = 1, 2 \ldots, n \tag{5}$$

Also of relevance to this approach is the angular component. SODA implements the concept of cosine distance, which is defined as Gu and Angelov (2018):

$$d_A(x_i, x_j) = \sqrt{1 - \cos(\Theta_{x_i, x_j})} i, j = 1, 2 \ldots, n \tag{6}$$

where $\cos(\Theta_{x_i, x_j}) = \frac{<x_i, x_j>}{\|x_i\| . \|x_j\|}$ denotes the value of the angle between x_i and x_j and which is also known as cosine similarity Gu and Angelov (2018). Our fuzzification approach implements the measurement and angular component values mutually calculated between the samples on a 2D plane, called direction-aware (DA) plane Gu and Angelov (2018).

As previously mentioned, the empirical data analytics (EDA) operators (Angelov et al. 2017b) work with the cloud definition. The relevant operators used here are the cumulative proximity (π), the local density (D_n), and the global density (D_n^G):

$$\pi_n(\boldsymbol{x}_i) = \sum_{j=1}^{n} d^2(\boldsymbol{x}_i, \boldsymbol{x}_j) \tag{7}$$

$$D_n(\boldsymbol{x}_i) = \frac{\sum_{j=1}^{n} \pi_n(\boldsymbol{x}_j)}{2n\pi_n(\boldsymbol{x}_i)} \tag{8}$$

$$D_n^G(\boldsymbol{\epsilon}_i) = f_i D_n(\boldsymbol{\epsilon}_i) \tag{9}$$

where $d(x_i, x_j)$ denotes the distance/dissimilarity between x_i and x_j.

As can be seen from the preceding equations, they can be updated recursively. Using the Euclidean component (Eq. 5) allows recursive calculation of the cumulative proximity (Eq. 7). This can be expressed by Gu and Angelov (2018):

$$\pi_n^M(\boldsymbol{x}_i) = \sum_{j=1}^{n} d_M^2(\boldsymbol{x}_i, \boldsymbol{x}_j) = n\left(\left\| \boldsymbol{x}_i - \boldsymbol{\mu}_n^M \right\|^2 + X_n^M - \left\| -\boldsymbol{\mu}_{\mathbf{n}}^{\mathbf{M}} \right\|^2 \right) \tag{10}$$

where $\boldsymbol{\mu}_n^M$ and X_n^M represent the mean of the data space and its squared norm mean, respectively. These two terms can be updated by Gu and Angelov (2018):

$$\boldsymbol{\mu}_n^M = \frac{n-1}{n} \boldsymbol{\mu}_{n-1}^M + \frac{1}{n} \boldsymbol{x}_n; \boldsymbol{\mu}_1^M = \boldsymbol{x}_1 \tag{11}$$

$$X_n^M = \frac{n-1}{n} X_{n-1}^M + \frac{1}{n} \left\| \boldsymbol{x}_n \right\|^2; X_1^M = \left\| \boldsymbol{x}_1 \right\|^2 \tag{12}$$

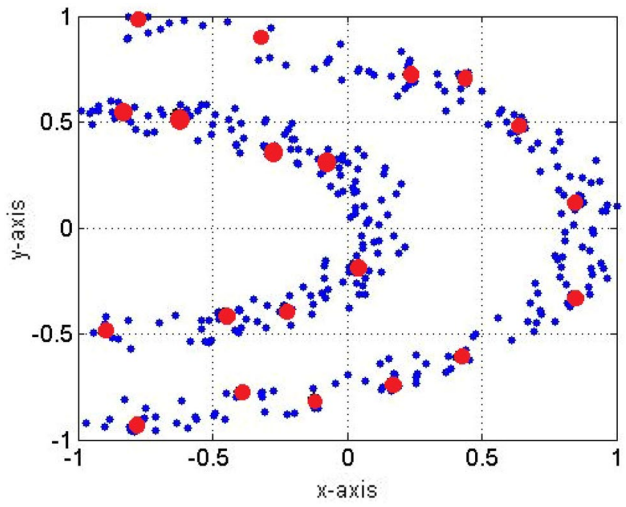


Fig. 2 Cloud centers (large, red dots) according to the SODA algorithm

The angular component updates similarly (Gu and Angelov 2018):

$$\boldsymbol{\mu}_n^A = \frac{n-1}{n}\boldsymbol{\mu}_{n-1}^A + \frac{1}{n}\frac{\boldsymbol{x}_n}{\|\boldsymbol{x}_n\|};\ \boldsymbol{\mu}_1^A = \frac{\boldsymbol{x}_1}{\|\boldsymbol{x}_1\|} \tag{13}$$

Using Euclidean distance to define the local density yields the following equation:

$$\sum_{j=1}^{n} \pi_n^M(x_j) = 2n^2\left(X_n^M - \left\|-\boldsymbol{\mu}_{\mathbf{n}}^{\mathbf{M}}\right\|^2\right) \tag{14}$$

This shows that the proposed SODA algorithm is able to process data streams in an incremental manner. In the SODA approach, understanding the concepts related to density is very helpful. This theme has been explored extensively by the authors in Angelov et al. (2017a, b). The local density D_n is expressed as the inverse of the normalized cumulative proximity and directly indicates the main pattern of the data observed (Angelov et al. 2017a), where D_n is defined as Gu and Angelov (2018):

$$D_n(x_i) = \frac{\sum_{j=1}^{n}\pi_n^M(x_j)}{2n\pi_n^M(x_i)} \tag{15}$$

Figure 2 shows an example of the SODA definition and the centers (red points) of the density grouping defined by the algorithm.

The SODA algorithm's key steps involve:

- first, form different DA planes of the recognized data samples, handling both the magnitude-based and angular-based densities;
- second, identify focal points;
- finally, use the focal points to partition the data space into data clouds Gu and Angelov (2018).

The algorithm comprises the following steps:

Stage 1—Preparation: Calculate the average values between every pair of data samples, $x_1, x_2, \ldots, x_n$ for both, the square Euclidean components d_M and the square angular components, d_A.

$$\begin{aligned}\bar{d}_M^2 &= \frac{\sum_{i=1}^{n}\sum_{j=1}^{n} d_M^2(\boldsymbol{x}_i, \boldsymbol{x}_j)}{n^2} \\ &= \frac{\sum_{i=1}^{n}\sum_{j=1}^{n}\left\|\boldsymbol{x}_i - \boldsymbol{x}_j\right\|^2}{n^2} = 2\left(X_n^M - \left\|-\boldsymbol{\mu}_{\mathbf{n}}^{\mathbf{M}}\right\|^2\right)\end{aligned} \tag{16}$$

$$\begin{aligned}\bar{d}_A^2 &= \frac{\sum_{i=1}^{n}\sum_{j=1}^{n} d_A^2(\boldsymbol{x}_i, \boldsymbol{x}_j)}{n^2} \\ &= \frac{\sum_{i=1}^{n}\sum_{j=1}^{n}\left\|\frac{\boldsymbol{x}_i}{\|\mathbf{x}_i\|} - \frac{\boldsymbol{x}_j}{\|\boldsymbol{x}_j\|}\right\|^2}{2n^2} = 1 - \left\|\boldsymbol{\mu}_n^A\right\|^2\end{aligned} \tag{17}$$

After calculating the global density, the approach reclassifies the samples' global density in a decreasing way, renaming them to $\{\hat{\boldsymbol{\epsilon}}_1, \hat{\boldsymbol{\epsilon}}_2, \ldots, \hat{\boldsymbol{\epsilon}}_{n_u}\}$.

Stage 2—DA plane projection: The DA projection operation begins with the unique data sample with the highest global density, namely u_1^-. It is initially set to be the first reference, $\mu_1 \leftarrow \hat{\boldsymbol{\epsilon}}_j$, which is also the origin of the first DA plane, denoted by P_1 ($L \leftarrow 1$, L is the number of DA planes existing in the data space). The following rule can be expressed as the second step of the algorithm:

$$\textbf{Condition 1}\ \begin{array}{c} IF\left(\frac{d_M(\boldsymbol{\mu}_l, \hat{\boldsymbol{\epsilon}}_j)}{\bar{d}_M} < \frac{1}{\gamma}\right) AND \left(\frac{d_A(\boldsymbol{\mu}_l, \hat{\boldsymbol{\epsilon}}_j)}{\bar{d}_A} < \frac{1}{\gamma}\right) \\ THEN\left(\mathbf{P}_l \leftarrow \hat{\boldsymbol{\epsilon}}_j\right)\end{array} \tag{18}$$

where γ assists on the decision of the granularity of the group used. Following the proposal in Gu and Angelov (2018), we used the value of $\gamma = 6$. When multiple DA planes meet Condition 1 at the same time, $\hat{\boldsymbol{\epsilon}}_j$ will be assigned to the closest one according to the following equation:

$$i = \underset{l=1,2,\ldots,L}{\arg\min}\left(\frac{d_M(\boldsymbol{\mu}_l, \hat{\boldsymbol{\epsilon}}_j)}{\bar{d}_M} + \frac{d_A(\boldsymbol{\mu}_l, \hat{\boldsymbol{\epsilon}}_j)}{\bar{d}_A}\right) \tag{19}$$

In this step, the mean, the support of the cloud, and the sum of the global density (mu_i, S_i and D_i, respectively) are updated as follows:

$$\boldsymbol{\mu}_i \leftarrow \frac{S_i}{S_i+1}\boldsymbol{\mu}_i + \frac{1}{S_i+1}\hat{\boldsymbol{\epsilon}}_j \tag{20}$$

$$S_i \leftarrow S_i + 1 \tag{21}$$

$$\mathbf{D}_i \leftarrow \mathbf{D}_i + D_n^G\left(\widehat{\boldsymbol{\epsilon}}_j\right) \tag{22}$$

However, if Condition 1 is not met, a new DA plane (P_{L+1}) is created by setting its initial parameters as follows:

$$\boldsymbol{\mu}_L \leftarrow \widehat{\boldsymbol{\epsilon}}_j \tag{23}$$

$$S_L \leftarrow 1 \tag{24}$$

$$\mathbf{D}_L \leftarrow D_n^G\left(\widehat{\boldsymbol{\epsilon}}_j\right) \tag{25}$$

Additionally, the number of overall clouds L is increased by 1:

$$L \leftarrow L + 1 \tag{26}$$

This process is repeated until all samples of the initial data set have been visited. Some data samples may be located on several DA planes simultaneously, depending on the nature of the problem. In this case, the final affiliation of these samples is decided by the distances between them and the origin of the closest DA plane (Gu and Angelov 2018).

Stage 3—Identifying the focal points: For each DA plane, denoted as P_e, find the neighboring DA planes ($\{\mathbf{P}\}_e^n, n = 1, 2, \ldots, L, n \neq e$). The condition for deciding to evolve new focal points can be defined by the following relationship:

$$\textbf{Condition 2}\ IF\left(\frac{d_M(\boldsymbol{\mu}_e,\boldsymbol{\mu}_l)}{\bar{d}_M} \leq \frac{2}{\gamma}\right) AND\left(\frac{d_A(\boldsymbol{\mu}_e,\boldsymbol{\mu}_l)}{\bar{d}_A} \leq \frac{2}{\gamma}\right) \\ THEN\left(\{\mathbf{P}\}_e^n \leftarrow \{\mathbf{P}\}_e^n \cup \left\{\mathbf{P}_l\right\}\right) \tag{27}$$

The main mode/peak of data density ($\mathbf{P_e}$) is determined if the following condition is met (where the corresponding $\mathbf{D}$ of $\{\mathbf{P}\}_e^n, n = 1, 2, \ldots, L, n \neq e$ is expressed by $\{\mathbf{D}\}_e^n$ Gu and Angelov (2018)):

$$\textbf{Condition 3}\ IF\left(\mathbf{D}_e > \max\left(\{\mathbf{D}\}_e^n\right)\right)\ THEN\ \left(\mathbf{P}_e\ is\ a\ mode/peak\ of\ \mathbf{D}\right) \tag{28}$$

All peaks have been found when Conditions 2 and 3 are met concurrently.

Stage 4—Forming data clouds: After all the DA planes for the modes/peaks of the data density have been identified, we define their origins, denoted by μ_o, as the focal points for forming data clouds according to a Voronoi tessellation (Okabe et al. 2009). The expression that represents the data cloud is given by Gu and Angelov (2018):

$$\textbf{Condition 4}\ IF\left(v = \underset{j=1,2,\ldots,C}{\arg\min}\left(\frac{d_M\left(\boldsymbol{x}_l,\boldsymbol{\mu}_j^o\right)}{\bar{d}_M} + \frac{d_A\left(\boldsymbol{x}_l,\boldsymbol{\mu}_j^o\right)}{\bar{d}_A}\right)\right) \\ THEN\ (\boldsymbol{x}_l\ is\ assigned\ to\ the\ v^{th}\ data\ cloud) \tag{29}$$

where C is the number of focal points.

Note that the concept of data clouds is similar to the concept of clusters, except that they are non-parametric, do not have a specific shape, and can express any real data distribution that follows a concept of local density (Angelov and Yager 2012).

In our evolving approach, some steps are necessary for new samples to cause changes in the data clouds found. The following steps, which are connected to the steps listed previously, assist in defining the update of the SODA cluster method:

Stage 5—Update parameters of existing clouds: For each new incoming sample $\boldsymbol{x}_n$, $\boldsymbol{\mu}_n^M$, $\boldsymbol{\mu}_n^A$ and X_n^M are updated by (11)), (12)), and (13)). Similarly, the Euclidean components and the angular components between the new sample, and the centers of the DA planes are also updated for each new sample using Eqs. (5) and (6), and are then represented by $d_M(\boldsymbol{x}_n, \boldsymbol{\mu}_l)$ and $d_A(\boldsymbol{x}_n, \boldsymbol{\mu}_l)$, for $l = 1, 2, \ldots, L$, respectively.

At this point in the process, the DA plane projection stage is triggered, allowing concurrent analysis of Condition 1 and Eq. (19) to determine whether the new sample belongs to an existing cluster (in this case, the model parameters are updated based on Eqs. (20) and (21)). If the new sample is unlike anything previously seen, a new DA plane is created using Eqs. (26), (23) and (24) Gu and Angelov (2018).

Stage 6—Fusion of overlapping DA planes: After the completion of Stage 5, the following condition is analyzed to identify strongly overlapping DA planes:

$$\textbf{Condition 5}\ IF\left(\frac{d_M(\boldsymbol{\mu}_i,\boldsymbol{\mu}_j)}{\bar{d}_M} < \frac{1}{2\gamma}\right) AND\left(\frac{d_A(\boldsymbol{\mu}_i,\boldsymbol{\mu}_j)}{\bar{d}_A} < \frac{1}{2\gamma}\right) \\ THEN\ (\mathbf{P}_i\ and\ \mathbf{P}_j\ are\ strongly\ overlapping) \tag{30}$$

Since such overlaps are to be avoided, a new DA plane based on P_j is created by merging the analyzed DA. This merger criterion is expressed by:

$$L \leftarrow L - 1 \tag{31}$$

$$\boldsymbol{\mu}_j \leftarrow \frac{S_j}{S_j+S_i}\boldsymbol{\mu}_j + \frac{S_i}{S_j+S_i}\boldsymbol{\mu}_i \tag{32}$$

$$S_j \leftarrow S_j + S_i \tag{33}$$

In the case of sample-wise updates from a stream, only the updated DA plane must be checked for significant overlaps with other DA planes. This process eliminates the parameters of the updated plane P_i and then returns to Stage 5.

Stage 7—Forming data clouds: As soon as all samples have been analyzed, the SODA model identifies all the focal points of the existing centers in the DA planes that resulted from Stage 6. The following steps are linked to the global density calculation of the centers of the DA plane by Eq. (9), using the support of each DA plane as the number of iterations to be performed. This yields the global density, $D_n^G(\boldsymbol{\mu}_l)$.

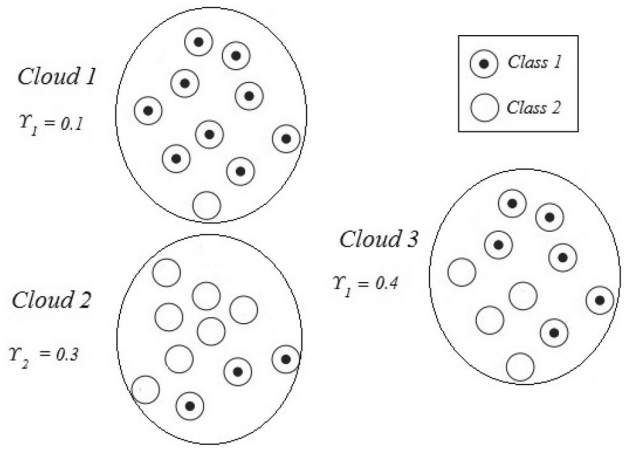


Fig. 3 Uncertainty of clusters in relation to their majority class

Each DA plane is then analyzed using Condition 2 to find its neighboring DA plan. After this, Condition 3 is used in order to verify whether $D_n^G(\boldsymbol{\mu}_e)$ is one of the local maxima of $(D_n^G(\boldsymbol{\mu}_l))$. Finally, all maximum $D_n^G(\boldsymbol{\mu}_l)$ locations identified and the centers of their corresponding DA planes are used to construct the focal points and consequently the data clouds.

At the beginning of the modelling stage, the SODA algorithm's behavior will resemble offline behavior (Stages 1–4). When new data is subsequently fed to the model, SODA acts in its evolving form, with a primary focus on the evolving stages (Stages 5–7).

The integration of label uncertainty is now done in the following way. As training is supervised, at the end of the SODA offline stage the number of samples that belong to each cloud can be checked. It is therefore possible to verify the uncertainty of a cloud in representing a particular class. This cloud uncertainty Υ_c for the majority class c, that is, the class which has the most samples over all classes within a cloud, is given by the ratio between class c samples and the total samples belonging to that cloud. Thus, for each class, a cloud that represents it better can be identified (that with the highest uncertainty in the respective class). This cluster uncertainty in the majority class can be calculated by:

$$\Upsilon_c = 1 - \frac{S_{cL}}{S_L} \tag{34}$$

where S_{cL} and S_L represent the number of samples belonging to the majority class and the total number of samples assigned to that cloud, respectively. Figure 3 illustrates uncertainty in clouds, showing three clouds with different majority classes. The uncertainty about the main class clouds is expressed by the number of samples that belong to Class 1 (first and third cloud) or Class 2 (second cloud) relative to the total number of samples belonging to each cloud.

In this work, we chose the class label uncertainty ψ_i in the ith sample as being (actively) informed by the user (e.g., she/he is not entirely confident in her/his annotation to a degree of ψ_i) or based on the user's experience with the process under study (typically estimated/defined in advance). Class label uncertainty is used to evaluate cluster allocation in online training (when the majority class c has the same label as the sample analyzed) under the following condition:

$$\textbf{Condition 6} \quad \begin{array}{c} IF\left(\psi_i < \Upsilon_c\right) \\ THEN(Start\ Stage\ 7) \end{array} \tag{35}$$

When the majority class of the cloud differs from the sample label, another closer cloud is sought to verify the Condition 6 in . If there is no closer cloud with majority class equal to the sample's class, the new sample forms a new cloud.

If the sample's degree of uncertainty is higher than the degree of uncertainty of the cloud to which it was assigned, another cloud in proximity is sought that has a lower degree of uncertainty than the sample (related to the same majority class). If in the new assessment of cloud and sample uncertainty Equation (35) is satisfied, Eqs. (12), (13) and (14) are applied again. If neither cloud fulfills this condition, the sample initiates a new cloud of which it becomes the center. To prevent the number of clouds from growing uncontrollably and thus impacting the fuzzy rules, an abnormal-cloud-exclusion approach was applied to detach new clouds from the model and control their complexity. This guarantees that a sample with a very high degree of uncertainty is excluded from the fuzzification process and does not interfere with the update of the data density, the global cloud density, and other essential elements.

The idea behind this approach is that, if a new sample is analyzed and its uncertainty is higher than that of the cloud it was initially assigned to, it is not assigned to in that cloud and is destined for another, neighboring cloud. For example, consider that a new sample submitted to the model has $\psi = 0.15$ and that it was initially attributed to Cloud 1 (Fig. 3). As its degree of uncertainty is high compared to that of the cloud ($0.15 > 0.1$), the sample will not be assigned to this cloud but to the closest one with the same majority class for which it was originally intended (Cloud 3 in this example). Assessing the uncertainties of Cloud 3 and of the sample using Eq. (35), shows that the sample should be assigned to this cloud. If the sample's ψ were 0.45, it would not meet the criteria established in Eq. (35) for the two clouds to which it could be assigned. In this case, this sample would trigger the evolution of a new cloud, but the exclusion criterion for abnormal clouds would be applied, which would exclude this sample (with a high degree of uncertainty) from the model's fuzzification process.

3.2 Second layer

The layer responsible for building fuzzy rules is composed by L fuzzy logic neurons extracted by the SSODA-U approach (as one cloud corresponds to one neuron). Each fuzzy logic neuron performs a weighted aggregation of all first-layer outputs. This aggregation is performed using the feature weights w_{ij} (for $i = 1 \ldots N$ and $j = 1 \ldots L$) as calculated by (4). For each input variable j, only one first-layer output a_{ij} is defined as input to the jth neuron. These neurons can extract knowledge from the data set and generate fuzzy rules. This type of neuron can be seen as a fuzzy logic equation, which allows extraction of knowledge in the form of IF-THEN rules (Pedrycz and Gomide 1998). In our approach, we use n-uninorms which allow the connections between rule antecedents to be represented by ANDs or ORs, offering greater interpretability than classical t-norms (which use only AND-connections).

The neurons in this layer use fuzzy operators to aggregate the first-layer elements. To this end, the t-norm (t or T) was used as a product and the t-conorm (s or S) as a probabilistic sum. The most traditional fuzzy neurons are the and-neuron and the or-neuron, which are constructed by:

$$\mathbf{z} = AND(\mathbf{w};\mathbf{a}) = T_{i=1}^{N}(w_i \; s \; a_i) \tag{36}$$

$$\mathbf{z} = OR(\mathbf{w};\mathbf{a}) = S_{i=1}^{N}(w_i \; t \; a_i) \tag{37}$$

where $\mathbf{a} = [a_1, a_2, \ldots, a_3, \ldots a_N]$ represents the Gaussian neurons in the first layer and $\mathbf{w} = [w_1, w_2, \ldots, w_3, \ldots w_N]$ are their respective weights, with N the number of inputs, thus $\mathbf{a}, \mathbf{w} \in [0,1]^N$.

Although they are useful for building fuzzy rules, they generate rules with the same antecedent connector (when using the and-neuron, only AND type connectors are generated in the rules). To create heterogenic fuzzy rule bases, that is, AND-rules and OR-rules, specific neurons termed unineuron (Lemos et al. 2010) and nullneuron (Hell et al. 2008) are employed. To aggregate neurons in the first layer of the model they use fuzzy operators, called uninorm (Yager and Rybalov 1996) and nullnorm (Calvo et al. 2001). These two types of fuzzy operators allow flexibility in creating fuzzy rules, as both can act as and-neuron or or-neuron, depending on the value of the identity element (uninorn) or the absorption element (nullnorm), which are both set in]0,1[. Despite having a similar structure, these two operators have different functions regarding the variation of identity and absorption elements. The closer to zero the identity element, the more the fuzzy operator resembles a t-norm to operate its functions (→ AND); the closer to 1, the more similar the operator becomes to a t-conorm (→ OR).

Studies have shown that these operators satisfy commutativity, monotonicity, associativity, and identity (Yager and Rybalov 1996). In this paper, the uninorm and nullnorm operators are represented by:

$$U(x,y,g) = \begin{cases} g\; T(\frac{x}{o}, \frac{y}{g}), & if\; y \in [0,g] \\ g + (1-g)\; S\,(\frac{x-g}{1-g}, \frac{y-g}{1-g}), & if\; y \in (g,1] \end{cases} \tag{38}$$

$$NU(x,y,u) = \begin{cases} uS(\frac{x}{u}, \frac{y}{u}), & if\; y \in [0,u] \\ u \;+\; (1-u)\; T\,(\frac{x-u}{y-u}, \frac{1-y}{1-u}), & if\; y \in (u,1] \end{cases} \tag{39}$$

Unineurons and nullneurons use first-layer input processing and their respective weights at two levels. At the first level, the input signals submitted to the structure are calculated individually using the weights. At the second level, the results of all first-level components are aggregated and transformed into unique values (p-function). These fuzzy operators use common processes in which each pair (a_i, w_i) is converted into a value $b_i = \mathbf{h}\,(a_i, w_i)$. At the second level, the aggregation of all previous values is calculated using a logic operator $\zeta\,(b_1, b_2 \ldots b_N)$, where ζ can be represented by unineurons (U) or nullneurons (NU). This operation can be represented mathematically by:

$$p(w,a,\chi) = wa + \bar{w}\chi \tag{40}$$

where χ is the identity or the absorption element, depending on the neuron used in the model, and $\bar{w}$ represents the complement of w.

The uni-nullneurons can be represented generically by:

$$z = N_{un-null}(w;a;\chi) = \zeta_{i=1}^{N} p(w_i, a_i, \chi) \tag{41}$$

These neurons facilitate the construction of more flexible fuzzy rules.

To leverage the dynamics of using these two operators together, the author in Akella (2007) proposed operators called n-uninorms, which generalize uninorm and nullnorm within the same fuzzy operator. Several n-uninorms meet these operators' specifics, (see Akella (2007)). However, here we use an operator called 2-uninorm, which allows the use of uninorms and nullnorms in the same operator.

Akella defined a function V: $[0,1]^2 \rightarrow [0,1]$ as a commutative binary function with $g, u, \beta \in [0,1]$ with $0 \le g \le \beta \le u \le 1$ and $0 < \beta < 1$. Therefore, $\{g,u\}_\beta$ is the definition of a 2-neutral element of V if it satisfies Akella (2007):

- V(g, x) =x for all x $\le \beta$
- V(u, x) =x for all x $\ge \beta$

The 2-neutral element defines the binary operator called 2-uninorm, $N^{un} = [0,1]^2 \rightarrow [0,1]$, if the operator is commutative, associative and has $\{g,u\}_\beta$ Zhou and Liu (2020).

The N^{un} operator can act as a uninorm if $g = \beta = u$, and can turn into a nullnorm if $g = 0$ and $u = 0$. This behavior of the 2-uninorm operator is flexible regarding its operators and, as $0 < g < \beta < u < 1$, it can be written as follows Zhou and Liu (2020):

$$N^{un}(x, y, \beta, g, u) = \begin{cases} \beta U_1(\frac{x}{\beta}, \frac{y}{\beta}), & if\ x, y \in [0, \beta] \\ \beta + (1-\beta)\, U_2\,(\frac{x-\beta}{1-\beta}, \frac{y-\beta}{1-\beta}), & if\ x, y \in (\beta, 1] \end{cases} \tag{42}$$

where U_1 is a uninorm (Eq. (3839)) with a neural element (identity) $= \frac{g}{\beta}$ and U_2 is a uninorm (Eq. (3839)) using $(\frac{u-\beta}{1-\beta})$ as a neutral element. The properties of this operator have been described in Zhou and Liu (2020). In general, it is possible to verify that this operator uses the identity and absorption elements and a third element to define which operations will be performed by the operator.

The uninull-neuron is a fuzzy neuron that uses a 2-uninorm as a fuzzy operator to build fuzzy rules. This neuron also has two-level processing, with the Gaussian neurons' inputs and their respective weights in the first layer. Therefore, based on Eq. (40), we can define the p-function as de Campos Souza and Lughofer (2022a):

$$p(w, a, \beta, g, u) = \begin{cases} wa + \bar{w}\frac{g}{\beta}, & if\ U_1 \\ wa + \bar{w}\frac{u-\beta}{1-\beta}, & if\ U_2 \end{cases} \tag{43}$$

Thus Eq. (41) can be adapted to represent the uninull-neuron as de Campos Souza and Lughofer (2022a):

$$z = UNI^{NUL}(w, a, \beta, g, u) = N^{un}_{i=1}{}^{N} p(w_i, a_i, \beta, g, u) \tag{44}$$

The fuzzy rules generated by uninull-neurons can be expressed by:

$$\begin{aligned} Rule_1 :\ & If x_{i1}\ is\ A_1^1\ with\ impact\ w_{11} \ldots \\ AND/OR_{(g,u,\beta)}\ & x_{i2}\ is\ A_1^2\ with\ impact\ w_{21} \ldots \\ AND/OR_{(g,u,\beta)}\ & x_{iN}\ is\ A_1^N\ with\ impact\ w_{N1} \ldots \\ & Then\ y_1\ is\ v_1 \\ & with\ average\ uncertainty\ \Psi_1 \\ & \ldots\ldots.. \\ Rule_L :\ & If x_{i1}\ is\ A_L^1\ with\ impact\ w_{1L} \ldots \\ AND/OR_{(g,u,\beta)}\ & x_{i2}\ is\ A_L^2\ with\ impact\ w_{2L} \ldots \\ AND/OR_{(g,u,\beta)}\ & x_{i2}\ is\ A_L^N\ with\ impact\ w_{NL} \ldots \\ & Then\ y_L\ is\ v_L \\ & with\ average\ uncertainty\ \Psi_L. \end{aligned} \tag{45}$$

The average uncertainty per rule Ψ_i can thus be calculated by the average uncertainty in the class labels of the samples which formed the rule, that is, for which their cloud representation was the closest, and thus its support S_i is incremented by 1 (see (21)):

$$\Psi_i = \frac{\sum_{i \in S_i} \psi_i}{S_i}. \tag{46}$$

where $v_1, \ldots, v_L$ represent the singleton consequents responsible for the (certainty of the) classification response of the corresponding rules: in our case of binary classification problems, these values are within [0, 1], where a value close to 0 represents a high certainty in Class 0 (and a low one in Class 1), and vice versa for a value close to 1. Their update based on stream samples considering uncertainty in the $\{0, 1\}$-class labels provided by the user is presented in the next section. Our rules can represent certainty degrees in the rule outputs based on uncertainty provided by the user. This achieves a higher level of interpretability given the user input on the problem. Rounding the consequent values to the nearest integer provides crisp $\{0, 1\}$ outputs.

The values of g, u, and β in the AND/OR connections of the rule antecedents define how the rules can finally be read. Further details on the combinations of the values and the corresponding neurons can be found in de Campos Souza and Lughofer (2022a).

3.3 Third layer

Finally, the output layer consists of one neuron (which can be considered as a singleton) whose activation function is given by:

$$y = \Omega\left(\sum_{j=0}^{L} f_\Gamma(z_j, v_j)\right) and\ \Omega = \begin{cases} 1, & if\ \sum_{j=0}^{L} f_\Gamma(z_j, v_j) > 0 \\ -1, & if\ \sum_{j=0}^{L} f_\Gamma(z_j, v_j) < 0 \end{cases} \tag{47}$$

where $z_0 = 1$, v_0 is the bias, and z_j and v_j, $k = 1, \ldots, L$, are the output and corresponding weight of each fuzzy neuron of the second layer, respectively, and f_Γ represents the neuron activation function.

The output neural network weights are obtained by the recursive least squares (RLS) approach as described below, which is extended with the integration of the label's uncertainty level.

3.4 Training method

In this sub-section, the definition of the consequents of the fuzzy rules (the v's), which also represent the weights that connect the second layer of the model to the neural aggregation network, is obtained in two stages, using the concepts of extreme learning machine (ELM) in the offline phase and a modified recursive least squares in the evolving stage. The latter was modified in a way such that the calculation of $\mathbf{v}$' is also influenced by the uncertainty in user-feedback labels.

In the offline training stage, weights are determined following the extreme learning machine approach (Huang et al. 2006b):

$$\mathbf{v} = \mathbf{Z}_{rul}^{+} y \tag{48}$$

where $\mathbf{Z}_{rul}^{+}$ is the Moore-Penrose pseudo-inverse (Albert 1972) of the activation matrix $\mathbf{Z}_{rul}$ ($(\mathbf{Z}_{rul}^{T}\mathbf{Z}_{rul})^{-1}$), which contains the activations of all neurons. This result is the least squares solution for the weights of the output layer. The $\mathbf{v}$'s values are updated on-line by using recursive least-squares (Huang et al. 2006a). This approach can be used to update the weights of the output layer as follows (Ljung 1999) (here from sample (time point) $t-1$ to sample t):

$$\eta = \mathbf{z}^t Q^{t-1} \left(\rho + (\mathbf{z}^t)^T Q^{t-1} \mathbf{z}^t\right)^{-1} \tag{49}$$

$$Q^t = (\mathrm{I}_{L^t} - \eta^T \mathbf{z}^t)\rho^{-1} Q^{t-1} \tag{50}$$

$$\mathbf{v}^t = \mathbf{v}^{t-1} + \eta^T (y^t - \mathbf{z}^t \mathbf{v}^{t-1}) \tag{51}$$

where $\mathbf{z}$ denotes the activation levels of all neurons in the current stream sample, η is the current kalman gain (row) vector, $I_{L_s^t}$ is an identity matrix based on the number of neurons (L) in the second layer, $L_s^t \times L_s^t$, Q denotes the inverse Hessian matrix $(\mathbf{Z}_{rul}^{T}\mathbf{Z}_{rul})^{-1}$ and is set initially to $\omega \mathrm{I}_{L_s^t}$, where ω=1000 as in Rosa et al. (2014), and is then continuously updated directly by Eq. (50) without requiring any re-inversion of matrices; ρ is a forgetting factor and typically set to values between 0.9 and 1.

An intuitive way of integrating the uncertainty information provided by the user, would be to use a fuzzy value that reflects the uncertainty degree and falls between the two crisp values of 0 and 1 for the two classes; for instance, if the user provided an uncertainty level of 0.2 together with class label 1, then y could be set to 0.8 (rather than to 1, which would be the classical case without uncertainty information), and then Eq. (51) would be used for updating. This would induce a movement of the v's away from 1 towards 0.8. However, the problem with such an integration is that it assumes reciprocal uncertainty for the two classes: this means that an uncertainty of 0.2 in Class label 1 automatically assumes an uncertainty of 0.8 in Class label 0. This may not necessarily be the case, as samples may be labeled with high uncertainty in both classes (e.g., 0.8 and 0.9), and the user chooses the less uncertain class as final label. In such a case, a full update would be conducted with a value close to 0, which would then incorrectly reflect a low uncertainty in class label 0. Therefore, we integrate the uncertainty in the class label of the current sample ψ_t in (51) resulting in the new approach termed as *RLS-U (recursive least squares with uncertainty integration)* as follows:

$$\mathbf{v}^t = \mathbf{v}^{t-1} + \eta^T (y^t - \mathbf{z}^t \mathbf{v}^{t-1})(1 - \psi_t) \tag{52}$$

Uncertainty is thus integrated by a multiplication of $(1-\psi_t)$ with the current prediction error (multiplied with Kalman gain). The effort for updating the current consequent vector v (relative to the error) is low when the uncertainty is high (independent of the label assigned) and high (or full) when the uncertainty is low (or zero as in the classical case). Samples with labels indicating a high uncertainty therefore have a little influence on the model's update, which is desired. The training algorithm for our ENFC-U approach can be summarized in Algorithm 1.

Algorithm 1 ENFC-U training and update algorithm

Initial Batch Learning Phase (Input: data matrix X with K samples):

(1) Extract L clouds in the first layer using the SSODA-U approach (where L is estimated automatically).
(2) Estimate center values $\mathbf{c}$ and sigmas $\boldsymbol{\sigma}$ for the L clouds derived from SSODA-U.
(3) Calculate the combination (feature) weights $\mathbf{w}$ for neuron construction using (4).
(4) Construct L logic neurons in the second layer of the network by welding the L fuzzy neurons of the first layer, using the uni-nullnorm concept according to (44) and centers $\mathbf{c}$ and widths $\boldsymbol{\sigma}$.
(5)
for $i = 1, ..., K$ **do**
  (5.1) Calculate the regression vector $z(x_i)$.
  (5.2) Store it as a one-row entry in the activation level matrix Z.
end for
(6) Estimate the weights of the output layer by (48) using Z_{sel} and indicator vectors $\mathbf{y}_k$.

Update Phase (Input: single data sample $(\mathbf{x}_t, y)$ with class label y and class label uncertainty ψ_t):

(7) Update L clouds and evolve a new one on demand (depending on rule evolution conditions) in the first layer, using the extended evolving SSODA-U approach and the class label uncertainty ψ_t provided by the user.
(8) Update the feature weights $\mathbf{w}$ by updating the within- and between-class scatter matrices and recalculating (4).
(9) Perform Steps (2) and (4).
(10) Calculate the regression vector $z(\mathbf{x}_t)$.
(11) Update the weights of the output layer by (52) using ψ_t and the class label y.

4 Results and discussion

4.1 Preliminary test definitions and evaluation criteria

In the experiments, various uncertainty levels were introduced in order to simulate users' certainty degrees on class labels and their experience levels. It is known that uncertainty can drastically affect the accuracy of data-driven and machine learning models (Lewis and Catlett 1994). Therefore, when introducing uncertainty in this context, the final impact on model accuracy should be ideally lower than the uncertainty level introduced by the user. Thus, we evaluated our approach by comparing the classification accuracy on data containing no uncertainty with the accuracy achieved on data including an associated uncertainty level of X%. Hence, when an average data uncertainty of around X% is introduced, the accuracy of the model obtained from the learning algorithm should decrease by less than X%.

In this work, we chose classification accuracy as stream evaluation criterion. This form of verification compares the result predicted by the model with that obtained in the experiments and can be expressed by:

$$Accuracy = \frac{TP + TN}{TP + FN + TN + FP} \tag{53}$$

where TP = the number of true positives, TN = the number of true negatives, FN = the number of false negatives and FP = the number of false positives. In the tests described below, accuracy was combined with a trend line approach to adapt it to the stream mining case. In this context, the one-step-ahead prediction accuracy is updated cumulatively by:

$$Accuracy(K+1) = \frac{Accuracy(K) * K + I_{\hat{y}=y}}{K+1}, \tag{54}$$

where I denotes the indicator function, which equals 1 when the prediction is correct, (i.e., $\hat{y} = y$) and 0 otherwise ($Accuracy(0) = 0$); subsequently, the model is updated. This results in an interleaved test-and-then-train scenario, which is widely used in the stream mining community, see Bifet et al. (2010). Note that for y we always use the given (ground

Table 1 Cyber-attack datasets

Dataset	Acronym	Source	Attacks	Normal commands
SQLDataset.data	SQLD	Demertzis and Iliadis (2015)	12.881	1.003
VirusDataset.data	MALD	Perdisci et al. (2008)	669	2.598

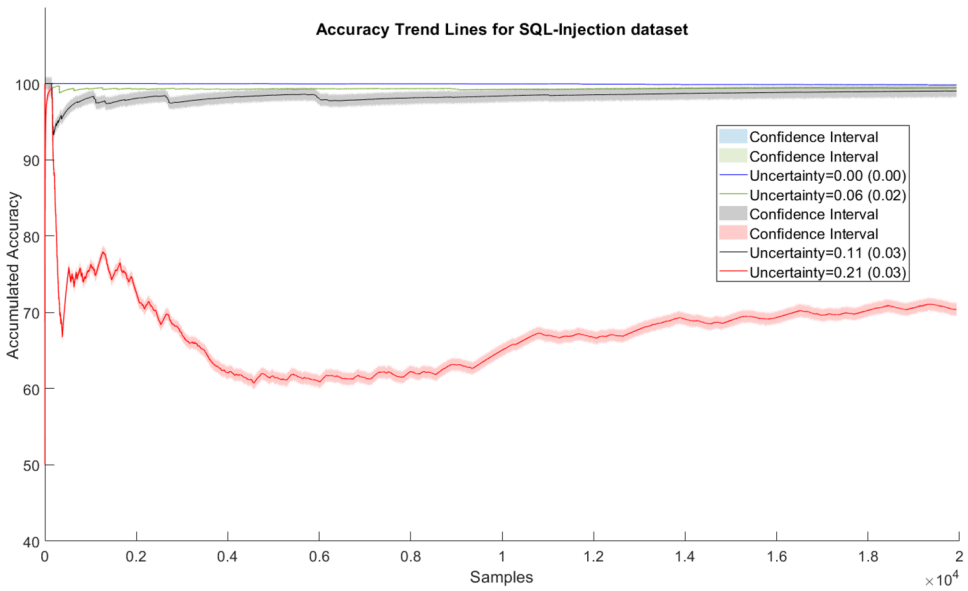


Fig. 4 Accuracy trend lines of SQL injection—SQLD data set

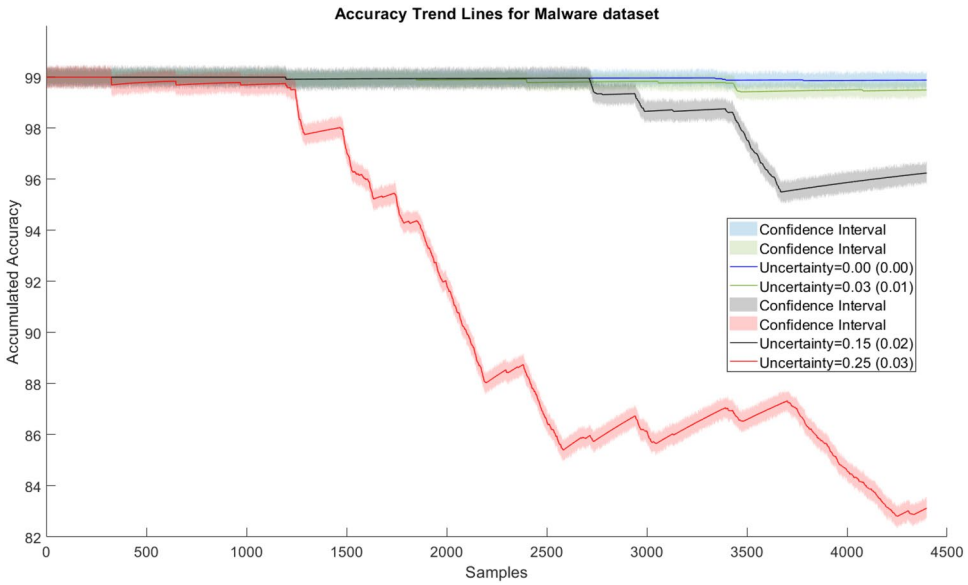


Fig. 5 Accuracy trend lines of Malware—MALD data set

truth) labels in the stream, and that the accuracy at each sample K can be stored in a vector, to create an accuracy trend line (as shown further below for various data streams).

All feature samples in the experiments were normalized such that they had a mean equal to zero and a standard deviation equal to one. The uncertainty provided by the user was sampled randomly. Uncertainty intervals were defined to identify model efficiency for various levels of sample uncertainty. Thus, we ran tests where the average uncertainty was zero, simulating the data in its original form, and tests with uncertainty in the ranges 0–10%, 10–20%, and 20–30% to simulate low, average and high sample uncertainty, respectively, in order to assess the (general) robustness of our method with respect to these uncertainty levels (i.e., to check how much the accuracy trend lines deteriorate in these cases).. Ten tests were performed on different shuffles of the data in each experiment in order to better visualize significant differences along the accumulated accuracy trends (as all were then surrounded by a one-sigma area among the ten runs). Furthermore, we performed statistical preference analysis tests based on the accuracy trends, in order to check for statistically significant differences in the trend lines.

Furthermore, in order to be able to check the performance of our extensions to SODA (SSODA-U) and RLS (RLS-U), where class label uncertainty is explicitly integrated (as discussed in the methodological section above), we compared

Table 2 Input features of the auction problem Alzahrani and Sadaoui (2018)

Feature	Description
Bidder tendency	An SB participates exclusively in auctions of few sellers rather than a diversified lot to help in this cheating
Bidding ratio	An SB participates more frequently to raise the auction price and attract higher bids from legal participants
Successive outbidding	An SB always outbids himself even though he is the current winner, slowly increasing the price with small values
Last bidding	An SB becomes inactive when the auction is near to 90% of its duration to avoid winning
Auction bids	auctions with dubious activities tend to have a much higher number of bids than the average
Auction starting price	An SB usually offers a modest starting price to invite legitimate bidders into the auction
Early bidding	An SB tends to bid in less than 25% of the auction duration to get the attention of auction users
Winning ratio	An SB competes in many auctions but rarely wins any
Auction duration	How long an auction remained
Class	0 for normal behaviour bidding; 1 otherwise

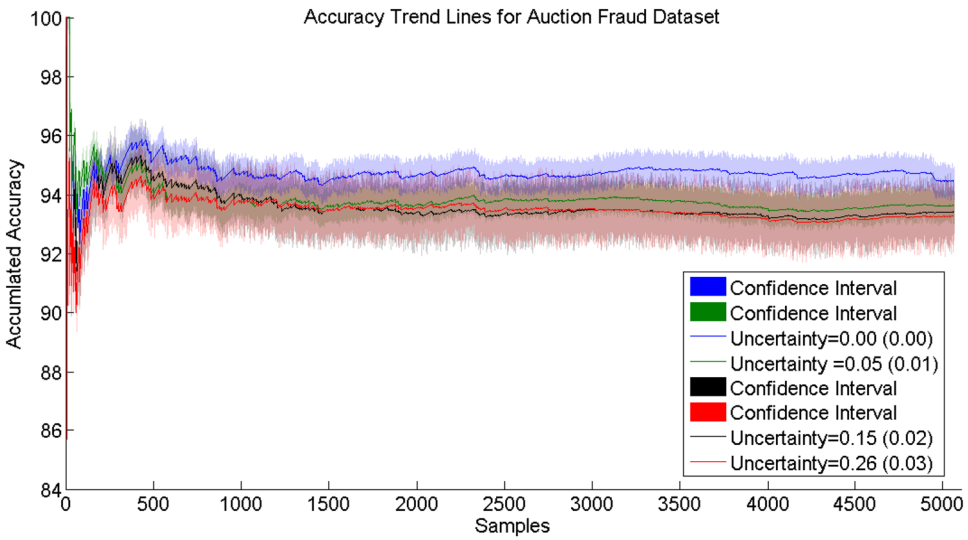


Fig. 6 Accuracy trend lines for auction fraud data sets

them with the original SODA and RLS variants (which do not consider uncertainty levels) on modified data streams from our application scenarios (see below), where we simulate X% uncertainty by changing the original (ground truth) class labels in X% of the samples and used the X% uncertainty level as ψ-values in our extensions (the original variants cannot use these ψ-values). This mimicked the case in which less experienced users label the data.

4.2 Cyber attack' identification tests

Several types of hacker actions can interfere with computers' proper functioning and systems that depend on the internet. Simulated attack data is commonly applied to hybrid models (Souza et al. 2020; Campos Souzaet al. 2020; Batista et al. 2019). Such data sets can help to detect anomalous behavior in the internet. In this experiment, two data sets were used, one simulating SQL injection and the other represents malware attacks. Table 1 shows the basic properties of the data used in the tests.

Table 3 *t*-Test for auction fraud data set test results with a 95% confidence level

Model	*t*-value	*p*-value
Uncertainty (0.00)	1.42	0.82*
Uncertainty (0.05)	0.83	0.54*
Uncertainty (0.15)	0.21	0.06*
Uncertainty (0.26)	0.07	0.05*

The accuracy trend lines as calculated by (54) are shown in Figs. 4 and 5 for SQL injection and malware data streams, respectively.

The results for SQL injection invasions demonstrate our approach's viability because, despite using data with considerable degrees of uncertainty (up to 11%), the performance was minimally impacted. Also, the loss in accuracy stands out when data uncertainty becomes significantly higher

Table 4 Interpretability with respect to (the degree of) changes in fuzzy neurons during the evolution phase without uncertainty

Rule 1 changed in 3 membership functions and by a degree of 0.01 with no consequent change
Rule 2 did not change with no consequent change
Rule 3 changed in 5 membership functions and by a degree of 0.06 with no consequent change

Table 5 Interpretability with respect to (the degree of) changes in fuzzy neurons during the evolution phase with uncertainty

Rule 1 changed in 2 membership functions and by a degree of 0.03 with no consequent change
Rule 2 did not change with no consequent change
Rule 3 changed in 5 membership functions and by a degree of 0.08 with no consequent change

(around 20%). The results for high uncertainty demonstrate the training method's difficulty in identifying new samples. The model's ability to maintain accuracy when identifying cyber attacks (99.6% versus 98.9%) for data uncertainties (0% versus 6%) was outstanding. In the case of the malware data stream, the difference in the accuracy between models using data with and without uncertainty was greater. Especially, in the case when the uncertainty became more than 20%, a significant down-trend could be observed (red line in the figures). This somehow provided us achievable bounds of our approach. For the lower uncertainty level cases, the accuracy loss was much lower than the uncertainty increase, e.g., (99.8% with 0% uncertainty versus 99.2 with 5% uncertainty, thus only 0.6% lower, although the uncertainty level was 5% higher). This proves that the model can well cope with data uncertainty up to some level in a robust manner. Or in other words, up to a certain level our approach is robust in incorporating user input on class label uncertainty into model training.

4.3 Auction fraud identification

Using a well-known data set, we present the initial rules extracted from the data sets and how (much) these change due to further evolution on a separate online set.

A commonly known problem in online commerce is auction fraud, particularly in shill bidding (SB) practices, where auxiliary accounts to increase the bids on products in active auctions. Analysis of this type of behavior is complex as it involves various uncertainties, which makes it a perfect problem for testing our approach. To detect these practices, Alzahrani and Sadaoui compiled a data set of harmful practices in auctions with 6321 samples including standard and fraudulent bids Alzahrani and Sadaoui (2018). The original data set has 12 input features, but for our purposes the dimensions related to personal identification (Record ID, Auction ID and Bidder ID), which are identification values and not values collected in the experiments, were irrelevant and thus removed. The remaining nine dimensions are listed in Table 2.

The tests to identify fraud in auctions were carried out comparing models based on data with 0% and an average of 5% uncertainty. The results are shown in Fig. 6.

The results demonstrate that our approach of including expert input on uncertainty in the data works well in the context of detecting auction fraud. The results are similar to those of approaches that use SVM (support vector machine) techniques to solve this problem (Anowar et al. 2018). To prove this statement, a *t*-test was performed where the reference value (0.940) was obtained in Anowar et al. (2018). The results are listed in Table 3[1]

Even with an average of 5% uncertainty, our results were on average only less than 1% less accurate than those of the model that was evolved on the data with 0% uncertainty. Interestingly, for this data stream the accuracy trend line decreased only minimally when increasing the average uncertainty level further to 15% and even to 26%, which emphasizes the robustness of our approach. Additionally, we analyzed the fuzzy rules extracted in the context of fraud identification. The cloud filter for samples with a high degree of uncertainty worked well, as both models ultimately created the same number of fuzzy rules, thus knowledge extraction was not affected by uncertainty. The interpretability of the rules is a significant factor in understanding how the model changes depending on the level of uncertainty and how it can generate knowledge. Tables 4 and 5 present the degree of change in the model without uncertainty and with uncertainty contained in the data, respectively, based on the structural changes tracking approach as proposed in de Campos Souza and Lughofer (2021).

There are differences between the connectives and the membership functions of the antecedents under the fuzzy rules generated. Thus, knowledge about auction fraud can be better interpreted, the impact of uncertainty on the fuzzy rules generated can be explored. The fuzzy rules generated without data uncertainty are presented below:

1. If Bidder Tendency is Small with impact 0.02 or Bidding Ratio is Small with impact 0.11 or Successive Outbidding is Small with impact 1.00 or Last Bidding is Small with impact 0.00 or Auction Bids is Small with impact 0.00 or Auction Starting Price is Small with impact 0.00 or Early Bidding is Small with impact 0.00 or Winning Ratio is Small with impact 0.04 or Auction Duration is

[1] (* means that the results are statistically equivalent with respect to a confidence interval of 95%).

Small with impact 0.00 then Auction Fraud is Yes with certainty 0.36 and average (rule) uncertainty of 0%.
2. If Bidder Tendency is Medium with impact 0.02 and Bidding Ratio is Medium with impact 0.11 and Successive Outbidding is Medium with impact 1.00 and Last Bidding is High with impact 0.00 and Auction Bids is High with impact 0.00 and Auction Starting Price is Medium with impact 0.00 and Early Bidding is High with impact 0.00 and Winning Ratio is Medium with impact 0.04 and Auction Duration is Medium with impact 0.00 then Auction Fraud is No with certainty 0.01 and average (rule) uncertainty of 0%.
3. If Bidder Tendency is Medium with impact 0.02 or Bidding Ratio is High with impact 0.11 or Successive Outbidding is Medium with impact 1.00 or Last Bidding is Medium with impact 0.00 or Auction Bids is Medium with impact 0.00 or Auction Starting Price is High with impact 0.00 or Early Bidding is Medium with impact 0.00 or Winning Ratio is High with impact 0.04 or Auction Duration is High with impact 0.00 then Auction Fraud is Yes with certainty 1.00 and average (rule) uncertainty of 0%.

The changes generated in the fuzzy rules when uncertainty was included demonstrate that our approach can update the Gaussian centers in the first layer, which brings new knowledge about the problem. The fuzzy rules generated by simulating data that incorporates user input are:

1. If Bidder Tendency is Small with impact 0.02 or Bidding Ratio is Small with impact 0.11 or Successive Outbidding is Small with impact 1.00 or Last Bidding is Small with impact 0.00 or Auction Bids is Small with impact 0.00 or Auction Starting Price is Small with impact 0.00 or Early Bidding is Small with impact 0.00 or Winning Ratio is Small with impact 0.04 or Auction Duration is Small with impact 0.00 then Auction Fraud is Yes with certainty 0.21 and data average (rule) uncertainty of 5%.
2. If Bidder Tendency is Medium with impact 0.02 or Bidding Ratio is Medium with impact 0.11 or Successive Outbidding is Medium with impact 1.00 or Last Bidding is High with impact 0.00 or Auction Bids is High with impact 0.00 or Auction Starting Price is Medium with impact 0.00 or Early Bidding is High with impact 0.00 or Winning Ratio is Medium with impact 0.04 or Auction Duration is Medium with impact 0.00 then Auction Fraud is No with certainty 0.00 and data average (rule) uncertainty of 5%.
3. If Bidder Tendency is Medium with impact 0.02 and Bidding Ratio is High with impact 0.11 and Successive Outbidding is Medium with impact 1.00 and Last Bidding is Medium with impact 0.00 and Auction Bids is Medium with impact 0.00 and Auction Starting Price is High with impact 0.00 and Early Bidding is Medium with impact 0.00 and Winning Ratio is High with impact 0.04 and Auction Duration is High with impact 0.00

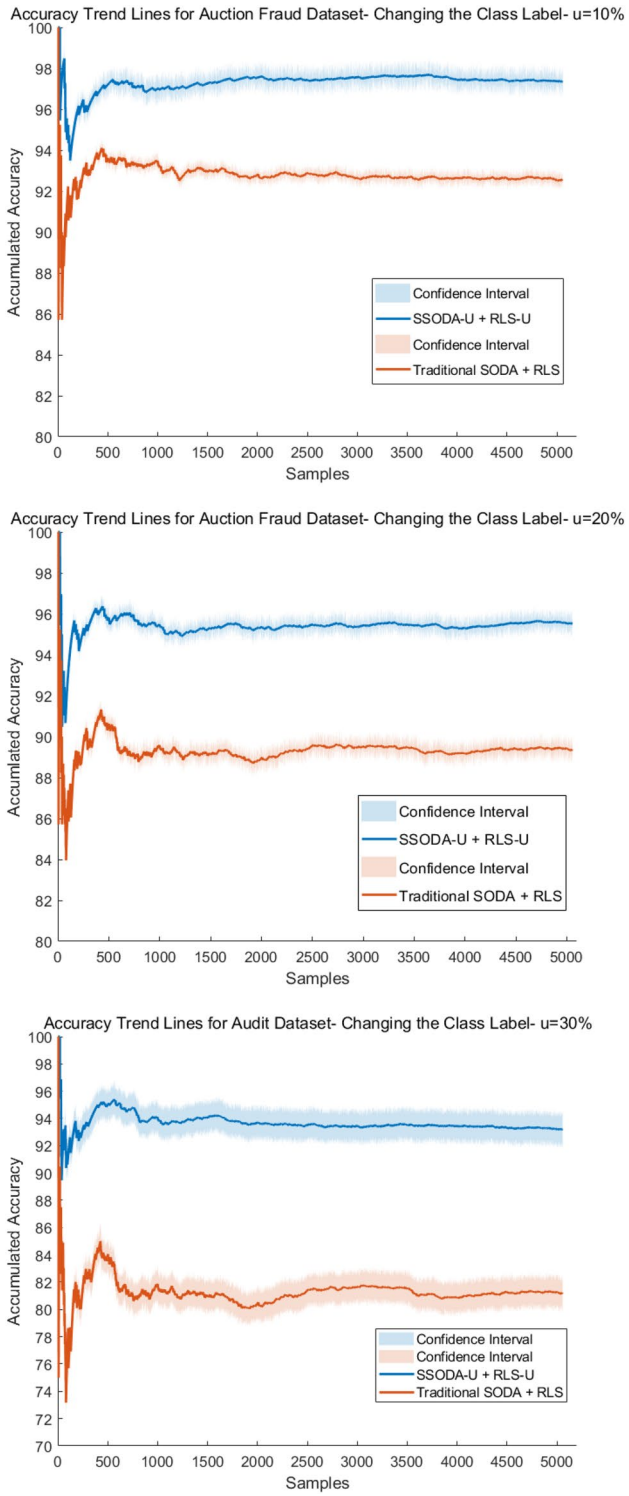


Fig. 7 Accuracy trend lines of auction fraud data set with X% changed labels

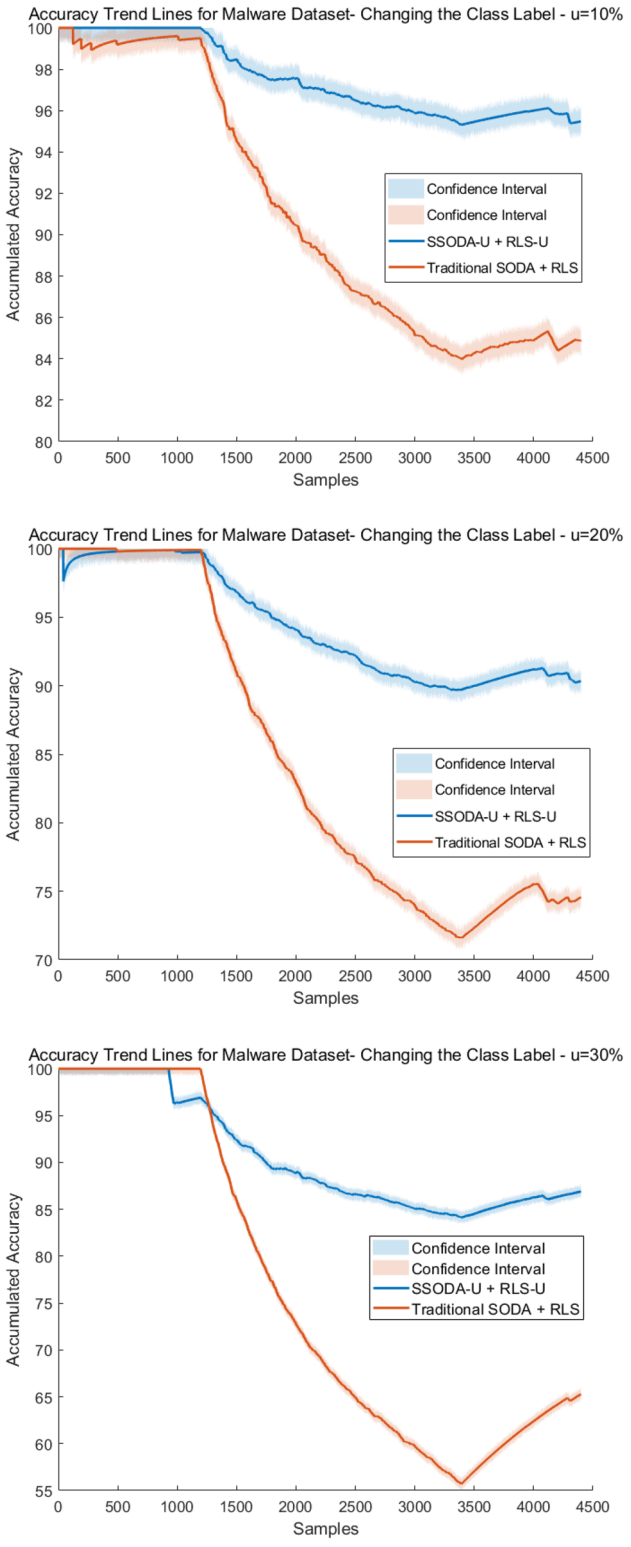


Fig. 8 Accuracy trend lines of malware data set with X% changed labels

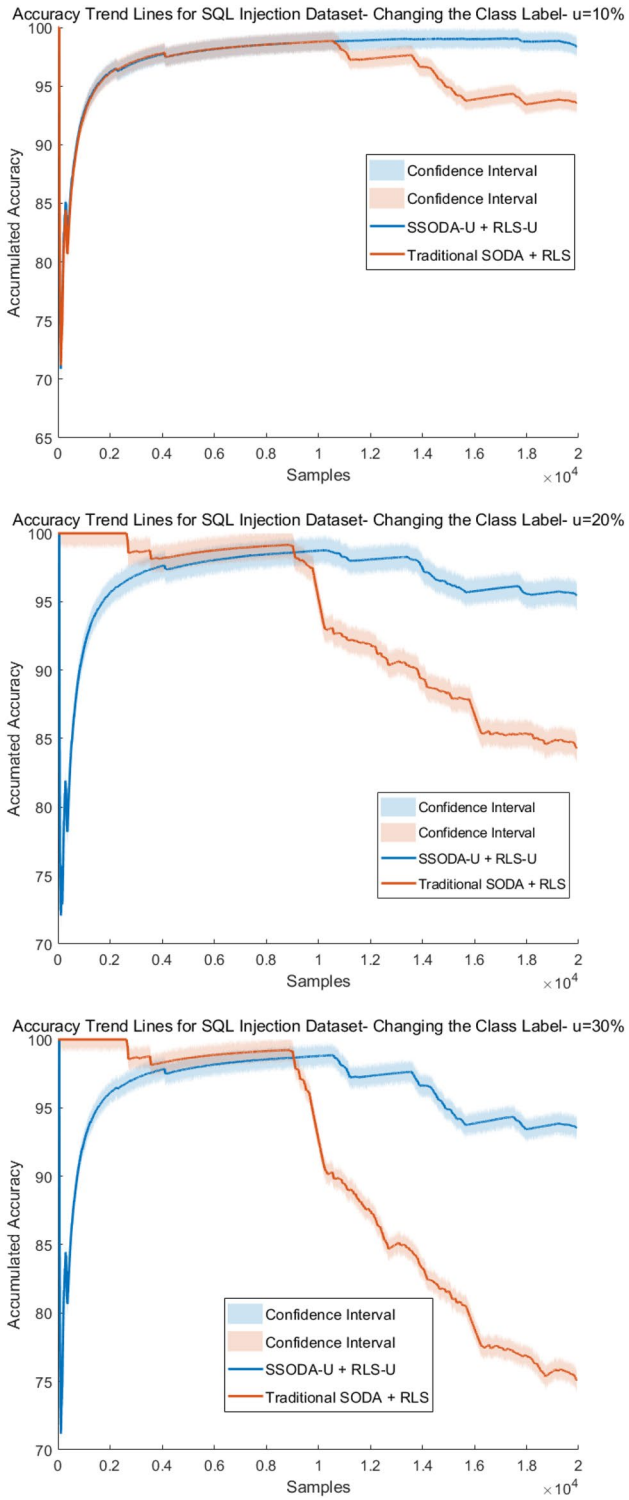


Fig. 9 Accuracy trend lines of SQL injection data set with X% changed labels

then Auction Fraud is Yes with certainty 1.00 and data average (rule) uncertainty of 5%.

Another factor to be highlighted is that the model was able to identify the most relevant feature in the auction fraud mode, which is the practice of successive outbidding. Other

Table 6 Assumptions of the ANOVA results: *p*-values

Test	Normality	Homoscedasticity	Independence
Auction fraud	0.0910	0.7114	0.4217
Malware	0.0598	0.5165	0.3130
Sql Injection	0.0741	0.4320	0.1220

features were rated as irrelevant (value 0) and some as being of low relevance. Thus, fuzzy rules can be seen as more straightforward and easy to understand by humans. An example of a fuzzy rule customized by taking into account the feature weights (features with weights of 0 can be omitted) is presented below:

If Bidder Tendency is Medium with impact 0.02 and Bidding Ratio is High with impact 0.11 and Successive Outbidding is Medium with impact 1.00 and Winning Ratio is High with impact 0.04 then Auction Fraud is Yes with certainty 1.00 and average (rule) uncertainty of 5%

If Bidder Tendency is Medium with impact 0.02 or Bidding Ratio is Medium with impact 0.11 or Successive Outbidding is Medium with impact 1.00 or Winning Ratio is Medium with impact 0.04 or then Auction Fraud is No with certainty 0.00 and average (rule) uncertainty of 5%.

This reduces the antecedent parts of the fuzzy rules from 9 down to 4, resulting in a much more readable rule base. Due to a comparison of the fuzzy rules, it is possible to verify the elements that favor one behavior over another. For the identification of fraud, the bidding ratio and winning ratio dimensions stand out. When they are present at high frequency, there is a greater likelihood of fraud. The first represents fraud with high certainty, and the second represents normal behavior in the auction with high likelihood.

4.3.1 Comparison tests with orginal model updates

In order to be able to check the performance of our extensions to SODA (SSODA-U) and RLS (RLS-U), which explicitly integrate class label uncertainty, we compared them with the original variants SODA and RLS (which do not consider uncertainty) on modified data streams from our application scenarios (see below), where we simulated X% uncertainty by changing the original (ground truth) class labels in X% of the samples and used the X% uncertainty level as ψ-values in our extensions (the original variants cannot use these ψ-values). This simulated the case in which an unexperienced user labels the data. We expected our new approach EFNC-U which uses SSODA-U and RLS-U, to be superior to the traditional SODA+RLS approach. Almost 10%, 20% and 30% of the sample labels were changed randomly. Therefore, we intend to compare whether the propositions made in this paper (an improvement on a previously published model) when evaluating the uncertainty of labels improves the model's efficiency in identifying the change of labels.

Figures 7, 8 and 9 illustrate the results.

It can be seen that, especially in the second phase of the streams, the accuracy trend lines differ significantly as they are affected by uncertainty, and that the behavior differs between the two approaches. For instance, for the auction fraud data set and 10% uncertainty, about 4% higher accuracies were achieved with our approach than with the

Table 7 Post-hoc Tukey test results for auction fraud data set with a 95% confidence level

Model–Model	Difference	Lower	Upper	*p*-value
SSODA-U+RLS-U—SODA+RLS (10%)	3.12	−0.77	2.12	0.03
SSODA-U+RLS-U—SODA+RLS (20%)	4.87	−1.18	3.49	0.02
SSODA-U+RLS-U—-SODA+RLS (30%)	5.39	−2.06	6.82	0.01

Table 8 Post-hoc Tukey test results for malware data set (MALD) with a 95% confidence level

Model–Model	Difference	Lower	Upper	*p*-value
SSODA-U+RLS-U—SODA+RLS (10%)	7.77	−0.81	4.82	0.02
SSODA-U+RLS-U—SODA+RLS (20%)	9.22	−1.19	6.31	0.01
SSODA-U+RLS-U—SODA+RLS (30%)	9.81	−1.82	8.20	0.00

Table 9 Post-hoc Tukey test results for SQL injection dataset (SQLD) with a 95% confidence level

Model–Model	Difference	Lower	Upper	*p*-value
SSODA-U+RLS-U—SODA+RLS (10%)	1.12	−0.19	1.98	0.05*
SSODA-U+RLS-U—SODA+RLS (20%)	5.21	−0.45	7.74	0.03
SSODA-U+RLS-U—SODA+RLS (30%)	9.39	−1.74	9.74	0.01

original SODA+RLS in the stream modeling process (compare the blue with the red line), while the difference is also noted to increase between the proposal of this paper and the traditional model (approximately 6% difference with 20% uncertainty and 10% for 30% uncertainty). In the case of the malware data set, the differences between the approaches were more significant and increasing (approximately 10%, 15% and 22% difference in accumulated accuracy for 10, 20 and 30% uncertainty respectively); this again shows high robustness with respect to uncertainty, and in all cases our approach outperformed traditional SODA+RLS. In the case of the SQL injection data set, the drop in the accuracy, concerning increasing uncertainty levels, during the test behavior is also highlighted as the uncertainty about the labels increases for traditional SODA+RLS.

Overall, our model achieved better accuracy results (in most cases and was at least on a par with) the traditional SODA model in the rest of the cases. This means that explicit integration of uncertainty levels in the update of the clouds and the consequent weight vectors resulted in a robust learning behavior. Such uncertainty levels can be used, for instance, in ML interaction/annotation systems when users with less experience than a super-user (who knows the ground truth) (start to) label the data.

The statistical analysis of variance (ANOVA) (St and Wold 1989) test was conducted to verify if the approaches proposed in these tests achieved the same performance when classifying each of the data sets with different levels of uncertainty. After its execution, it was found that the models did not have similar performances in any of the test groups (as achieving a p-value of 0.021 for the auction fraud test, a p-value of 0.007 for the malware test, and a p-value of 0.029 for the SQL injection test).

To verify the assumptions of the ANOVA results, three tests were used to confirm it: normality, homoscedasticity, and independence of data (Montgomery 2017). We confirmed the data normality conditions using the Shapiro–Wilk test. The homoscedasticity that verifies the equality of variances of the residues was validated using the Fligner–Killeen test, resulting in similar variances in all the classification performances. Finally, the Durbin–Watson test confirmed that the data under analysis is independent. All the results are listed in Table 6.

Thus, we conclude with 95% confidence that the models have different accuracy performances in each of the tests performed.

Once we verified the specific performances in the models, we conducted a post-hoc Tukey test for multiple comparisons, one-to-one, among all the models involved in the benchmark. The results are listed in Tables 7, 8 and 9 (* means that the results are statistically equivalent with respect to a confidence interval of 95%). Based on these results, the conclusion is, with 95% confidence, that the model proposed in this paper has a better behavior to understand the changes in the labels, and it can evolve with a higher performance than the related SoA method (conventional SODA+RLS).

5 Conclusion

ENFC-U, the evolving fuzzy neural classifier (ENFC) presented in this paper, incorporates user feedback on class label uncertainty in model training, adaptation and evolution. The results from cyber-attack tests and fraud assessment in auctions confirm the idea that it is possible to incorporate this uncertainty, while maintaining model accuracy of the problem analyzed. Furthermore, the model tests showed that there is a limit for the uncertainty level regarding robustness of the models predictions (measured in terms of accuracy trend lines). When the average uncertainty associated with the data was higher than 20%, the model lost its ability to correctly identify data labels, as its predictive performance was significantly deteriorated. Therefore, in addition to being a relevant factor for training the model, it also becomes its weakness because the levels of accuracy fall to levels greater than the uncertainty associated with the samples. The results are, however, robust when the uncertainty in the provided labels is less than 20%, as then the effect on the accuracy trend lines was negligible. The fuzzy rules generated by the evolving fuzzy neural network model contributed reliably to the identification of the classification problems, thus offering some sort of useful knowledge extraction. The generated fuzzy rules showed good interpretability, as they could be greatly reduced by its length due to the integration of feature weights indicating the impact of features on the classification problem. This factor allows a synergy between the model proposed in this paper and the user experience in adequately evaluating the samples involved in a problem evaluation process. By associating uncertainty with fuzzy rules consequents, experts can gain a basis for how strong a rule is favoring a certain class among the others in that particular region it (geometrically) represents. Furthermore, the uncertainty levels in the class labels did hardly affect the knowledge finally extracted, which indicates another form of robustness of our method.

Future work will be related to the incorporation of user uncertainty into other procedures, for instance, by setting weights for fuzzy neurons or other training forms. Further, it would be interesting to incorporate another type of user feedback into the model's update procedure, that is not related to the data, but, for example, to the output of the model or to its input structure (typically in form of expert rules). Another possible extension to this work would be to evaluate the data uncertainty in the definition of synaptic weights.

Acknowledgements The authors acknowledge the support by the Austrian Science Fund (FWF): contract number P32272-N38, acronym IL-EFS.

Funding Open access funding provided by Austrian Science Fund (FWF).

Data Availability This study was a re-analysis of existing data. The Auction fraud identification dataset analysed is available in the UCI Machine Learning Repository: https://archive.ics.uci.edu/ml/datasets/Shill+Bidding+Dataset. The Virus Data set is available in the Researchgate: https://www.researchgate.net/publication/328391682_virus_full. In the case of data sets referring to SQL Injection, it was obtained upon request from the researcher Konstantinos Demertzis, author of the paper that deals with this data set (Demertzis and Iliadis 2015).

Declarations

Conflict of interest The authors have no competing interests to declare.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Aggarwal CC, Philip SY, Han J, Wang J (2003) A framework for clustering evolving data streams. In: Proceedings 2003 VLDB conference. Elsevier, pp 81–92

Akella P (2007) Structure of n-uninorms. Fuzzy Sets Syst 158(15):1631–1651

Albert A (1972) Regression and the Moore-Penrose pseudoinverse. Elsevier, Amsterdam

Al-Turjman F, Zahmatkesh H, Mostarda L (2019) Quantifying uncertainty in internet of medical things and big-data services using intelligence and deep learning. IEEE Access 7:115,749–115,759, https://doi.org/10.1109/ACCESS.2019.2931637

Alzahrani A, Sadaoui S (2018) Scraping and preprocessing commercial auction data for fraud classification. arXiv preprint arXiv:1806.00656

Angelov P, Gu X (2018) Deep rule-based classifier with human-level performance and characteristics. Inf Sci 463–464:196–213

Angelov P, Yager R (2012) A new type of simplified fuzzy rule-based system. Int J Gen Syst 41(2):163–185

Angelov PP, Zhou X (2008) Evolving fuzzy-rule-based classifiers from data streams. IEEE Trans Fuzzy Syst 16(6):1462–1475. https://doi.org/10.1109/TFUZZ.2008.925904

Angelov P, Lughofer E, Zhou X (2008) Evolving fuzzy classifiers using different model architectures. Fuzzy Sets Syst 159(23):3160–3182

Angelov P, Gu X, Kangin D (2017a) Empirical data analytics. Int J Intell Syst 32(12):1261–1284

Angelov PP, Gu X, Príncipe JC (2017b) A generalized methodology for data analysis. IEEE Trans Cybern 48(10):2981–2993

Anowar F, Sadaoui S, Mouhoub M (2018) Auction fraud classification based on clustering and sampling techniques. In: 2018 17th IEEE international conference on machine learning and applications (ICMLA). IEEE, pp 366–371

Batista LO, de Silva GA, Araújo VS, Araújo VJS, Rezende TS, Guimarães AJ, Souza PVdC (2019) Fuzzy neural networks to create an expert system for detecting attacks by sql injection. Int J Forensic Comput Sci 13(1):8–21

Bifet A, Holmes G, Kirkby R, Pfahringer B (2010) MOA: massive online analysis. J Mach Learn Res 11:1601–1604

Bi J, Zhang T (2005) Support vector classification with input data uncertainty. In: Advances in neural information processing systems, pp 161–168

Calvo T, De Baets B, Fodor J (2001) The functional equations of frank and alsina for uninorms and nullnorms. Fuzzy Sets Syst 120(3):385–394

Cao F, Estert M, Qian W, Zhou A (2006) Density-based clustering over an evolving data stream with noise. In: Proceedings of the 2006 SIAM international conference on data mining. SIAM, pp 328–339

Casalino G, Castellano G, Mencar C (2019) Data stream classification by dynamic incremental semi-supervised fuzzy clustering. Int J Artif Intell Tools 28(08):1960009

Chatfield C (1995) Model uncertainty, data mining and statistical inference. J R Stat Soc A Stat Soc 158(3):419–444

Chen Y, Tu L (2007) Density-based clustering for real-time stream data. In: Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining, pp 133–142

D'angelo G, Palmieri F, Ficco M, Rampone S (2015) An uncertainty-managing batch relevance-based approach to network anomaly detection. Appl Soft Comput 36:408–418

de Campos Souza PV (2020) Fuzzy neural networks and neuro-fuzzy networks: a review the main techniques and applications used in the literature. Appl Soft Comput 92(106):275. https://doi.org/10.1016/j.asoc.2020.106275

de Campos Souza PV, Lughofer E (2021) An evolving neuro-fuzzy system based on uni-null neurons with advanced interpretability capabilities. Neurocomputing 451:231–251. https://doi.org/10.1016/j.neucom.2021.04.065

de Campos Souza PV, Lughofer E (2022a) An advanced interpretable fuzzy neural network model based on uni-nullneuron constructed from n-uninorms. Fuzzy Sets Syst 426:1–26. https://doi.org/10.1016/j.fss.2020.11.019

de Campos Souza PV, Lughofer E (2022b) Efnn-nulluni: An evolving fuzzy neural network based on null-uninorm. Fuzzy Sets Syst. https://doi.org/10.1016/j.fss.2022.01.010

de Campos Souza PV, Rezende TS, Guimaraes AJ, Araujo VS, Batista LO, da Silva GA, Silva Araujo VJ (2019) Evolving fuzzy neural networks to aid in the construction of systems specialists in cyber attacks. J Intell Fuzzy Syst 36(6):6743–6763

de Campos Souza PV, Guimarães AJ, Rezende TS, Souza Araujo V, do Nascimento LAF, Oliveira Batista L (2020) An intelligent hybrid model for the construction of expert systems in malware detection. In: 2020 IEEE conference on evolving and adaptive intelligent systems (EAIS), pp 1–8

de Campos Souza PV, Guimarães AJ, Rezende TS, Silva Araujo VJ, Araujo VS (2020) Detection of anomalies in large-scale cyberattacks using fuzzy neural networks. Artif Intell 1(1):92–116

de Campos Souza PV, Guimaraes Nunes CF, Guimares AJ, Silva Rezende T, Araujo VS, Silva Arajuo VJ (2021a) Self-organized direction aware for regularized fuzzy neural networks. Evol Syst 12(2):303–317

de Campos Souza PV, Soares EA, Guimarães AJ, Araujo VS, Araujo VJS, Rezende TS (2021b) Autonomous data density pruning

fuzzy neural network for optical interconnection network. Evol Syst 12(4):899–911
de Campos Souza PV, Lughofer E, Guimaraes AJ (2021c) An interpretable evolving fuzzy neural network based on self-organized direction-aware data partitioning and fuzzy logic neurons. Appl Soft Comput 112(107):829. https://doi.org/10.1016/j.asoc.2021.107829
Demertzis K, Iliadis L (2015) A bio-inspired hybrid artificial intelligence framework for cyber security. In: Computation, cryptography, and network security. Springer, pp 161–193
Dong XL, Halevy A, Yu C (2009) Data integration with uncertainty. VLDB J 18(2):469–500
Duan X, Wang Y, Pedrycz W, Liu X, Wang C, Li Z (2018) Afsnn: a classification algorithm using axiomatic fuzzy sets and neural networks. IEEE Trans Fuzzy Syst 26(5):3151–3163
Dy JG, Brodley CE (2000) Feature subset selection and order identification for unsupervised learning. In: Proceedings of the seventeenth international conference on machine learning,. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, ICML '00, pp 247–254
Dy J, Brodley C (2004) Feature selection for unsupervised learning. J Mach Learn Res 5:845–889
Gacto M, Alcala R, Herrera F (2011) Interpretability of linguistic fuzzy rule-based systems: An overview of interpretability measures. Inf Sci 181(20):4340–4360
Gama J (2010) Knowledge discovery from data streams. Chapman & Hall/CRC, Boca Raton, Florida
Ge C, Shi Y, Park JH, Hua C (2020) State estimate for fuzzy neural networks with random uncertainties based on sampled-data control. J Franklin Inst 357(1):635–650
Ghahramani Z (2015) Probabilistic machine learning and artificial intelligence. Nature 521(7553):452–459
Gu X, Angelov PP (2018) Self-organising fuzzy logic classifier. Inf Sci 447:36–51
Guimarães AJ, de Campos Souza PV, Lughofer E (2020) Hybrid model for parkinson's disease prediction. In: International conference on information processing and management of uncertainty in knowledge-based systems. Springer, pp 621–634
Hamidzadeh J, Ghadamyari R (2020) Clustering data stream with uncertainty using belief function theory and fading function. Soft Comput 24:8955–8974
Hariri RH, Fredericks EM, Bowers KM (2019) Uncertainty in big data analytics: survey, opportunities, and challenges. J Big Data 6(1):44
Hell M, Gomide F, Costa P (2008) Neurons and neural fuzzy networks based on nullnorms. In: 2008 10th Brazilian symposium on neural networks, pp 123–128
Hodges S, Moore P (1972) Data uncertainties and least squares regression. J Roy Stat Soc: Ser C (Appl Stat) 21(2):185–195
Huang GB, Chen L, Siew CK et al (2006a) Universal approximation using incremental constructive feedforward networks with random hidden nodes. IEEE Trans Neural Netw 17(4):879–892
Huang GB, Zhu QY, Siew CK (2006b) Extreme learning machine: theory and applications. Neurocomputing 70(1–3):489–501
Hühn J, Hüllermeier E (2009) FR3: a fuzzy rule learner for inducing reliable classifiers. IEEE Trans Fuzzy Syst 17(1):138–149
Huijbregts MA, Norris G, Bretz R, Ciroth A, Maurice B, von Bahr B, Weidema B, de Beaufort AS (2001) Framework for modelling data uncertainty in life cycle inventories. Int J Life Cycle Assess 6(3):127
Kasabov N (2001) Evolving fuzzy neural networks for supervised/unsupervised online knowledge-based learning. IEEE Trans Syst Man Cybern B (Cybern) 31(6):902–918
Kasabov N (2007) Evolving connectionist systems: the knowledge engineering approach, 2nd edn. Springer Verlag, London
Kirk PD, Stumpf MP (2009) Gaussian process regression bootstrapping: exploring the effects of uncertainty in time course data. Bioinformatics 25(10):1300–1306
Leite D, Ballini R, Costa P, Gomide F (2012) Evolving fuzzy granular modeling from nonstationary fuzzy data streams. Evol Syst 3(2):65–79
Leite D, Costa P, Gomide F (2013) Evolving granular neural networks from fuzzy data streams. Neural Netw 38:1–16
Lemos A, Caminhas W, Gomide F (2010) New uninorm-based neuron model and fuzzy neural networks. In: 2010 annual meeting of the North American fuzzy information processing society. IEEE, pp 1–6
Lewis DD, Catlett J (1994) Heterogeneous uncertainty sampling for supervised learning. In: Machine learning proceedings 1994. Elsevier, pp 148–156
Ljung L (1999) System identification: theory for the user. Prentice Hall, Upper Saddle River, New Jersey
Lughofer E (2011) On-line incremental feature weighting in evolving fuzzy classifiers. Fuzzy Sets Syst 163(1):1–23
Lughofer E, Buchtala O (2013) Reliable all-pairs evolving fuzzy classifiers. IEEE Trans Fuzzy Syst 21(4):625–641
Montgomery DC (2017) Design and analysis of experiments. John Wiley & Sons, New Jersey
Okabe A, Boots B, Sugihara K, Chiu SN (2009) Spatial tessellations: concepts and applications of Voronoi diagrams, vol 501. John Wiley & Sons, New Jersey
Pedrycz W, Gomide F (1998) An introduction to fuzzy sets: analysis and design. MIT Press, Cambridge
Pedrycz W, Izakian H (2014) Cluster-centric fuzzy modeling. IEEE Trans Fuzzy Syst 22(6):1585–1597
Perdisci R, Lanzi A, Lee W (2008) Mcboost: Boosting scalability in malware collection and analysis using statistical classification of executables. In: 2008 Annual computer security applications conference (ACSAC). IEEE, pp 301–310
Pratama M, Lu J, Lughofer E, Zhang G, Anavatti S (2016) Scaffolding type-2 classifier for incremental learning under concept drifts. Neurocomputing 191:304–329
Rosa R, Gomide F, Dovzan D, Skrjanc I (2014) Evolving neural network with extreme learning for system modeling. In: 2014 IEEE conference on evolving and adaptive intelligent systems (EAIS), pp 1–7
Schnute J (1987) Data uncertainty, model ambiguity, and model identification. Nat Resour Model 2(2):159–212
Se S, Lowe D, Little J (2002) Mobile robot localization and mapping with uncertainty using scale-invariant visual landmarks. Int J Robot Res 21(8):735–758. https://doi.org/10.1177/027836402761412467
Siler W, Buckley J (2005) Fuzzy expert systems and fuzzy reasoning: theory and applications. John Wiley & Sons, Chichester, West Sussex
Škrjanc I, Iglesias JA, Sanchis A, Leite D, Lughofer E, Gomide F (2019) Evolving fuzzy and neuro-fuzzy approaches in clustering, regression, identification, and classification: a survey. Inf Sci 490:344–368. https://doi.org/10.1016/j.ins.2019.03.060
Solomatine DP, Shrestha DL (2009) A novel method to estimate model uncertainty using machine learning techniques. Water Resources Res 45(12)
Souza PVdC (2018) Pruning fuzzy neural networks based on unineuron for problems of classification of patterns. J Intell Fuzzy Syst 35(2):2597–2605
Souza PV, dos Reis AG, Marques GRR, Guimaraes AJ, Araujo VJ, Araujo VS, Rezende TS, Batista LO, da Silva GA (2019a) Using hybrid systems in the construction of expert systems in the identification of cognitive and motor problems in children and young people. In: 2019 IEEE international conference on fuzzy systems (FUZZ-IEEE). IEEE, pp 1–6

Souza PVdC, Guimares AJ, Rezende TS, Araujo VS, Araujo VJS, Batista LO (2019b) Bayesian fuzzy clustering neural network for regression problems. 2019 IEEE international conference on systems, man and cybernetics (SMC). IEEE, pp 1492–1499

Souza P, Ponce H, Lughofer E (2020) Evolving fuzzy neural hydrocarbon networks: a model based on organic compounds. Knowl-Based Syst 203(106):099

Souza PVdC, Guimaraes AJ, Araujo VS, Lughofer E (2021) An intelligent bayesian hybrid approach to help autism diagnosis. Soft Comput 25(14):9163–9183

St L, Wold S et al (1989) Analysis of variance (anova). Chemom Intell Lab Syst 6(4):259–272. https://doi.org/10.1016/0169-7439(89)80095-4

Subramanian K, Das AK, Sundaram S, Ramasamy S (2014) A metacognitive interval type-2 fuzzy inference system and its projection based learning algorithm. Evol Syst 5(4):219–230

Wang L, Ji HB, Jin Y (2013) Fuzzy passive-aggressive classification: a robust and efficient algorithm for online classification problems. Inf Sci 220:46–63

Watson DF (1981) Computing the n-dimensional delaunay tessellation with application to voronoi polytopes. Comput J 24(2):167–172

Xu Y, Fang X, Li X, Yang J, You J, Liu H, Teng S (2014) Data uncertainty in face recognition. IEEE Trans Cybern 44(10):1950–1961. https://doi.org/10.1109/TCYB.2014.2300175

Yager RR, Rybalov A (1996) Uninorm aggregation operators. Fuzzy Sets Syst 80(1):111–120

Zadeh L (1965) Fuzzy sets. Inf Control 8(3):338–353

Zhang D, Pal SK (2000) A fuzzy clustering neural networks (fcns) system design methodology. IEEE Trans Neural Netw 11(5):1174–1177

Zhang C, Oh SK, Fu Z, Pedrycz W (2020) Self-organized hybrid fuzzy neural networks driven with the aid of probability-based node selection and enhanced input strategy. Neurocomputing 417:471–489

Zhou H, Liu X (2020) Characterizations of (u2, n)-implications generated by 2-uninorms and fuzzy negations from the point of view of material implication. Fuzzy Sets Syst 378:79–102

Zhou H, Zhao H, Zhang Y (2020) Nonlinear system modeling using self-organizing fuzzy neural networks for industrial applications. Appl Intell 50:1657–1672

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.